

Fundamentos de Organización de Datos – Curso 2025

Índice

Índice.....	1
Bibliografía.....	3
1) Archivos.....	3
Persistencia de datos.....	3
Propiedades implícitas de una BD.....	3
Definiciones de archivo.....	3
Hardware.....	3
Organización de un archivo.....	4
Acceso de archivos.....	4
Tipos de archivos.....	4
Operaciones con archivos.....	4
Archivos maestro y detalle.....	6
Merge.....	9
Archivos de longitud fija.....	12
Archivos de longitud variable.....	12
Fragmentación.....	14
Bajas.....	15
Organización de datos.....	18
Búsqueda de información y manejo de índices.....	19
2) Árboles.....	24
Árbol binario.....	24
Árbol AVL.....	24
Paginación de árboles binarios.....	25
Árbol multcamino.....	26
Árbol B.....	26
Árbol B+.....	28
Árbol B*.....	31
Buffers y árboles B virtuales.....	34
Conclusiones.....	35
3) Dispersión.....	36
Definiciones.....	36
Características.....	36
Densidad de empaquetamiento.....	37
Problemas.....	38

Técnicas de resoluciones.....	39
Conclusión general.....	45
Archivos (Organización Básica: Ninguna o Secuencial).....	45
Archivos Secuenciales Indizados (Árboles como Índices).....	46
Dispersión (Hashing).....	46

Bibliografía

- [Introducción a las Bases de Datos. Conceptos básicos \(Bertone, Thomas\).](#)
- [Estructuras de Archivos \(Folk-Zoellick\).](#)
- [Files & Databases: An introduction \(Smith-Barnes\).](#)

1) Archivos

Persistencia de datos

Una **base de datos** es una colección de datos relacionados, específicamente de archivos diseñados para servir a múltiples aplicaciones. Estos datos representan hechos conocidos que pueden registrarse y que tienen un resultado implícito.

Propiedades implícitas de una BD

Una base de datos...

1. ...representa algunos aspectos del mundo real, a veces denominado Universo de Discurso.
2. ...es una colección coherente de datos con significados inherentes. Un conjunto aleatorio de datos no puede considerarse una BD. O sea los datos deben tener cierta lógica.
3. ...se diseña, construye y completa de datos para un propósito específico. Está destinada a un grupo de usuarios concretos y tiene algunas aplicaciones preconcebidas en las cuales están interesados los usuarios.
4. ...está sustentada físicamente en archivos en dispositivos de almacenamiento persistente de datos.

Definiciones de archivo

Un archivo es una colección de...

1. ...registros guardados en almacenamiento secundario.
2. ...datos almacenados en dispositivos secundarios de memoria.
3. ...registros que abarcan entidades con un aspecto común y originadas para algún propósito particular.

Hardware

1. Almacenamiento primario: Memoria RAM
2. Almacenamiento secundario (DR): platos, superficies, pistas, sectores, cilindros.

Organización de un archivo

1. En una **secuencia de bytes**: archivos de texto dónde se leen o recuperan caracteres sin formato previo. Una palabra se determina por un conjunto de caracteres finalizados en blanco (convención).
2. En **registros y campos**: un campo es la unidad más pequeña lógicamente significativa de un archivo; un registro es un conjunto de campos agrupados que definen un elemento del archivo.

Acceso de archivos

1. Secuencial físico: acceso a los registros uno tras otro y en el orden físico en el que están guardados (sin orden específico, simplemente en cómo llegaron).
2. Secuencial indizado (lógico): acceso a los registros de acuerdo al orden establecido por otra estructura o criterio.
3. Directo: se accede a un registro determinado sin necesidad de haber accedido a los predecesores.

Tipos de archivos

Se determinan de acuerdo a su forma de acceso:

1. Serie: cada registro es accesible solo luego de procesar su antecesor, simples de acceder (acceso secuencial físico).
2. Secuencial: los registros son accesibles en orden de alguna clave (acceso secuencial indizado/secuencial lógico).
3. Directo: se accede al registro deseado (acceso directo).

Operaciones con archivos

Los **archivos físicos** aparecen en el disco y están a cargo del SO. En cambio, los **archivos lógicos** se definen dentro del programa. El archivo se puede definir de dos formas:

- Como variable:
`Var archivo: file of Tipo_de_dato;`
- Como tipo:
`Type archivo: file of Tipo_de_dato;
Var arch: archivo;`

Relación con el SO: se debe asignar la correspondencia entre el nombre físico y el lógico:

```
Assign(n_logico, n_fisico);
```

Rewrite: de solo escritura (creación):

```
Rewrite(n_logico);
```

Reset: lectura/escritura (apertura del archivo):

```
Reset(n_logico);
```

Close: cierre de archivo. Luego del ultimo dato del archivo se coloca la marca EOF (End Of file), es decir, al final del archivo:

```
Close(n_logico);
```

Read: leer un archivo sobre una variable del mismo tipo del archivo:

```
Read(n_logico, variable);
```

Write: escribo en el buffer:

```
Write(n_logico, variable);
```

Estas operaciones (read y write) leen y/o escriben sobre los buffers relacionados a los archivos No se realizan directamente sobre la memoria secundaria.

Ejemplo 1: generar archivo

```
program Generar_Archivo;
type archivo = file of integer; {definición del tipo de dato para el archivo }
var
  arc_logico: archivo; {variable que define el nombre lógico del archivo}
  nro: integer; {nro será utilizada para obtener la información de
  teclado}
  arc_fisico: string[12]; {utilizada para obtener el nombre físico del
archivo
desde teclado}

begin
  write( 'Ingrese el nombre del archivo:' );
  read( arc_fisico ); { se obtiene el nombre del archivo}
  assign( arc_logico, arc_fisico );
  rewrite( arc_logico ); { se crea el archivo }
  read( nro ); { se obtiene de teclado el primer valor }
  while nro <> 0 do begin
    write( arc_logico, nro ); { se escribe en el archivo cada número }
    read( nro );
  end;
  close( arc_logico ); { se cierra el archivo abierto oportunamente con la
instrucción
  rewrite }
end.
```

EOF: fin del archivo:

```
EOF(n_logico); // true o false
```

FileSize: tamaño del archivo:

```
FileSize(n_logico); // tamaño del archivo
```

FilePos: posición dentro del archivo:

```
FilePos(n_logico); // posición (integer)
```

Seek: ir a una posición del archivo. Siempre se cuenta desde el comienzo del archivo (principio = 0):

```
Seek(n_logico, posicion); // procedimiento
```

Ejemplo 2: agregar datos a un archivo existente

```
procedure agregar (var emp: empleados);
var
    e:registro;
begin
    reset(emp);
    seek(emp, filesize(emp)); // se posiciona al final del archivo
    leer(e);
    while(e.nombre <> ' ') do begin
        write(emp, e);
        leer(e);
    end;
    close(emp);
end;
```

Archivos maestro y detalle

La relación entre estos archivos debe ser compatible para poder actualizar el maestro a partir del detalle. En general, hay **un archivo maestro por cada N archivos detalle**.

Actualización de un archivo maestro con un archivo detalle

Caso 1)

- Existe un archivo maestro.
- Existe un único archivo detalle que modifica al maestro.
- Cada registro del detalle modifica a un registro del maestro. Esto significa que solamente aparecerán datos en el detalle que se correspondan con datos del maestro. Se descarta la posibilidad de generar altas en ese archivo.
- No todos los registros del maestro son necesariamente modificados.
- Cada elemento del maestro que se modifica es alterado por uno y solo un elemento del archivo detalle.
- Ambos archivos están ordenados por igual criterio. Esta precondition, considerada esencial, se debe a que hasta el momento se trabaja con archivos de datos de acuerdo con su orden físico. Más adelante, se discutirán situaciones donde los archivos respetan un orden lógico de datos.

Caso 2)

- Existe un archivo maestro.
- Existe un único archivo detalle que modifica al maestro.
- Cada registro del detalle modifica a un registro del maestro. Esto significa que solamente aparecerán datos en el detalle que se correspondan con datos del maestro. Se descarta la posibilidad de generar altas en el maestro.

- Cada elemento del archivo maestro puede no ser modificado, o ser modificado por uno o más elementos del detalle.
- Ambos archivos están ordenados por igual criterio. Esta precondition, considerada muy fuerte, se debe a que hasta el momento se manipulan archivos de datos de acuerdo con su orden físico. Más adelante, se discutirán situaciones donde los archivos respetan un orden lógico de datos.

Estos casos no son únicos.

Ejemplo 1: actualizar maestro a partir de detalle

```
program actualizar;
type
    emp = record
        nombre:string[30];
        direccion:string[30];
        cht:integer;
    end;
    e_diario = record
        nombre:string[30];
        cht:integer;
    end;
    detalle = file of e_diario;
    maestro = file of emp;
var
    regm:emp; regd:e_diario;
    mae1:maestro; det1:detalle;
begin
    assign(mae1,'maestro');
    assign(det1,'detalle');
    reset(mae1);
    reset(det1);
    while(not eof(det1)) do begin
        read(mae1,regm);
        read(det1,regd);
        while(regm.nombre <> regd.nombre) do
            read(mae1,regm);
        regm.cht := regm.cht + regd.cht;
        seek(mae1, filepos(mae1)-1);
        write(mae1,regm);
    end;
end.
```

Ejemplo 2: actualizar maestro a partir de detalle con corte de control

Cada bloque del archivo maestro contiene a lo sumo una estructura de datos (un registro, por ejemplo) de sus correspondientes archivos detalles (**precondición 3**), por lo que no puede haber información repetida en el maestro. Para actualizar el maestro sin repetir información de los detalles se utilizarán cortes de control:

```
program actualizar;
const VALOR_ALTO=9999;
type
    str4 = string[4];
```

```

prod = record
  cod:str4;
  descripcion:string[30];
  pu:real;
  cant:integer;
end;
v_prod = record
  cod:str4;
  ov:integer; {cantidad vendida}
end;
detalle = file of v_prod;
maestro = file of prod;
var
  regm:prod; regd:v_prod;
  mael:maestro; det1:detalle;
  total:integer;
begin
  assign(mae1,'maestro');
  assign(det1,'detalle'); {proceso principal}
  reset(mae1);
  reset(det1);
  while (not EOF(det1)) do begin
    read(mae1,regm);
    read(det1,regd);
    while (regm.cod <> regd.cod) do
      read(mae1,regm);
    while(not EOF(det1) and (regm.cod = regd.cod)) do begin
      regm.cant := regm.cant - regd.cv;
      read(det1,regd); // (!)
    end;
    if(not EOF(det1))
      seek(det1, filepos(det1)-1);
      seek(mae1, filepos(mae1)-1);
      write(mae1,regm);
    end;
  end;
end.

```

(!) Nótese que si estoy ante el ultimo registro del archivo, al evaluar aquel **while** el **read** ya avanzó y me perdería este último registro. Para abstraerse de esta situación, podemos crear un procedimiento **leer(archivoX,registroX)**:

```

procedure leer(var archivo:detalle; var dato:v_prod);
begin
  if(not EOF(archivo)) then
    read(archivo,dato)
  else
    dato.cod := VALOR_ALTO;
  end;
end;

```

Y en el programa principal suprimimos **not (EOF(det1))**:

```

...
while(regm.cod = regd.cod)) do begin
  regm.cant := regm.cant - regd.cv;
  leer(det1,regd); // (OK)
end;

```


...

Múltiples archivos detalle

Las precondiciones y la declaración de tipos son los mismos. El problema es que al tener N archivos detalles (cada uno ordenado), se hace más complejo encontrar el primer elemento entre todos (el menor) para actualizar su respectivo en el archivo maestro. Por ejemplo, no puedo actualizar un elemento y luego uno menor que este mismo. Para ello, se deben recorrer los N archivos detalle y encontrar el elemento mínimo para arrancar a actualizar su archivo maestro a partir de él.

```
begin // Programa principal
  assign(mae1, 'maestro'); assign(det1, 'detalle1');
  assign(det2, 'detalle2'); assign(det3, 'detalle3');
  reset(mae1); reset(det1); reset(det2); reset(det3);
  leer(det1, regd1); leer(det2, regd2); leer(det3, regd3);
  minimo(regd1, regd2, regd3, min);
  while (min.cod <> valoralto) do begin
    read(mae1, regm);
    while (regm.cod <> min.cod) do
      read(mae1, regm);
    while (regm.cod = min.cod ) do begin
      regm.cant:=regm.cant - min.cantvendida;
      minimo(regd1, regd2, regd3, min);
    end;
    seek (mae1, filepos(mae1)-1);
    write(mae1, regm);
  end;
end.
```

Merge

Implica fusionar o unir archivos con contenido similar.

Precondiciones

1. Todos los archivos detalle tienen la misma estructura.
2. Todos los archivos están ordenados por el mismo criterio.

Estas precondiciones no son únicas. Sin embargo, aseguran eficiencia en performance/rendimiento.

1. Fusión sin repeticiones

Ejemplo: CADP inscribe a los alumnos que cursarán la materia en tres computadoras separadas. C/U de ellas genera un archivo con los datos personales de los estudiantes, luego son ordenados físicamente por otro proceso. El problema que tienen los JTP es genera un archivo maestro de la asignatura.

```
procedure minimo (var r1,r2,r3:alumno; var min:alumno);
begin
  if (r1.nombre<r2.nombre) and (r1.nombre<r3.nombre) then begin
    min := r1;
    leer(det1,r1)
  end else if (r2.nombre<r3.nombre) then begin
    min := r2;
```

```

        leer(det2,r2)
end else begin
    min := r3;
    leer(det3,r3)
end;
end;

begin
    assign (det1, 'det1');
    assign (det2, 'det2');
    assign (det3, 'det3');
    assign (maestro, 'maestro');
    rewrite (maestro);
    reset (det1); reset (det2); reset (det3);

    leer(det1, regd1); leer(det2, regd2); leer(det3, regd3);
    minimo(regd1, regd2, regd3, min);

    { se procesan los tres archivos }
    while (min.nombre <> valoralto) do
begin
    write (maestro,min);
    minimo(regd1, regd2, regd3,min);
end;
    close (maestro);
end.

```

2. Fusión con repeticiones

En este ejemplo, los vendedores de cierto comercio asientan las ventas realizadas. Sin embargo, el archivo maestro deberá resumir los archivos detalle en código de vendedor y monto total entre todas las ventas. Es decir, los detalles y el maestro tienen diferente estructura (distintos registros).

```

begin
    assign (det1, 'det1');
    assign (det2, 'det2');
    assign (det3, 'det3');
    assign (mae1, 'maestro');
    reset (det1); reset (det2); reset (det3);
    rewrite (mae1);

    leer (det1, regd1); leer (det2, regd2); leer (det3, regd3);
    minimo (regd1, regd2, regd3, min);
    { se procesan los archivos de detalles }
    while (min.cod <> valoralto) do begin
        {se asignan valores para registro del archivo maestro}
        regm.cod := min.cod;
        regm.total := 0;
        {se procesan todos los registros de un mismo vendedor}
        while (regm.cod = min.cod ) do begin
            regm.total := regm.total+ min.montoVenta;
            minimo (regd1, regd2, regd3, min);
        end;
        { se guarda en el archivo maestro}
        write(mae1, regm);
    end;
end;

```

end.

3. Fusión de N archivos con repeticiones

```
program union_de_N_archivos;
const valoralto = '9999';
type
  vendedor = record
    cod: string[4];
    producto: string[10];
    montoVenta: real;
  end;
  ventas = record
    cod: string[4];
    total: real;
  end;
  maestro = file of ventas;
  arc_detalle = array[1..100] of file of vendedor; // asumir como válido
  reg_detalle = array[1..100] of vendedor; // ídem
var
  min: vendedor;
  deta: arc_detalle;
  reg_det: reg_detalle;
  mae1: maestro;
  regm: ventas;
  i,n: integer;
procedure leer (var archivo:detalle; var dato:vendedor);
begin
  if (not eof( archivo )) then
    read (archivo, dato)
  else
    dato.cod := valoralto;
  end;
procedure minimo (var reg_det: reg_detalle; var min:vendedor; var
deta:arc_detalle);
var i: integer;
begin
  { busco el mínimo elemento del
  vector reg_det en el campo cod,
  supongamos que es el índice del for es i }
  min = reg_det[i];
  leer(deta[i], reg_det[i]);
end;
begin
  Read(n)
  for i:= 1 to n do begin
    assign (deta[i], 'det'+i);
    { ojo lo anterior es incompatible en tipos }
    reset( deta[i] );
    leer( deta[i], reg_det[i] );
  end;
  assign (mae1, 'maestro'); rewrite (mae1);
  minimo (reg_det, min, deta);
  { se procesan los archivos de detalles }
  while (min.cod <> valoralto) do begin
    {se asignan valores para registro del archivo maestro}
```

```

    regm.cod := min.cod;
    regm.total := 0;

    {se procesan todos los registros de un mismo vendedor}
    while (regm.cod = min.cod ) do begin
        regm.total := regm.total+ min.montoVenta;
        minimo (regd1, regd2, regd3, min);
    end;

    { se guarda en el archivo maestro}
    write(mae1, regm);
end;
end.

```

Archivos de longitud fija

Un archivo de longitud fija es una colección de registros donde todos los elementos de datos almacenados son del mismo tipo y, por lo tanto, del mismo tamaño. La longitud de cada registro es predecible y está determinada por la suma de las longitudes de sus campos, lo que significa que el tamaño de cada elemento está preestablecido.

Características y Ejemplos

- La información es homogénea, es decir, todos los elementos son del mismo tipo.
- La longitud de cada elemento es conocida de antemano. Por ejemplo, un número entero puede ocupar 2 bytes, un carácter 1 byte, y una cadena de 20 caracteres (String) ocupará 20 bytes. Un registro con nombre (String), DNI (String) y edad (integer) ocuparía 50 bytes (40 + 8 + 2).
- Las operaciones de lectura y escritura se realizan de forma estructurada, con solo definir el tipo de dato del archivo.

Ventajas

- El proceso de entrada y salida de información desde y hacia los buffers es responsabilidad del sistema operativo, lo que simplifica la programación.
- Los procesos de alta, baja y modificación de datos se corresponden con las operaciones básicas sobre archivos.
- Permiten el uso de un Número Relativo de Registro (NRR) para calcular directamente la distancia en bytes desde el principio del archivo a cualquier registro, lo que facilita el acceso directo a la información con un solo acceso a disco.

Desventajas

- Tienden a generar fragmentación interna, lo que ocurre cuando se asigna más espacio del necesario a un elemento de dato (por ejemplo, un campo String para un nombre corto como "Juan Perez" solo usa 10 de los 50 bytes asignados). Este espacio reservado no utilizado representa un desperdicio.

Archivos de longitud variable

Los archivos de longitud variable son una forma de organizar la información donde la cantidad de espacio utilizada por cada registro no está determinada de antemano. Esta organización busca optimizar el uso del espacio en disco, utilizando solo el espacio necesario para almacenar la información.

Características y Funcionamiento

- Se conciben como una secuencia de caracteres.
- Cada elemento de dato debe descomponerse y guardarse carácter a carácter en el archivo (e.g., un string se guarda carácter a carácter, un dato numérico cifra a cifra).
- Requieren el uso de marcas que delimiten cada elemento, ya sean marcas de fin de campo o marcas de fin de registro. Estas marcas son caracteres especiales que no pueden formar parte de los datos.
- Las operaciones de lectura y escritura son más minuciosas, ya que el programador debe resolver la transferencia carácter a carácter.
- Pueden utilizar indicadores de longitud al principio de cada campo o registro, o un segundo archivo para mantener la información de la dirección de inicio de cada registro.

Ventajas

- La utilización de espacio en disco es optimizada, ya que cada registro ocupa solo el espacio que realmente necesita, a diferencia de los archivos de longitud fija.

Desventajas

- La implementación requiere un algoritmo donde el programador debe resolver de forma más minuciosa las operaciones de agregar y quitar elementos.
- El proceso de modificación es más complejo, especialmente si el nuevo registro ocupa más espacio que el anterior, lo que puede requerir reubicar el registro en el archivo. Esto a menudo se resuelve dando de baja el elemento anterior y dando de alta el nuevo registro.
- Tienden a generar fragmentación externa, que es el espacio disponible entre dos registros pero que es demasiado pequeño para ser reutilizado.
- Cuando se reutiliza el espacio, un nuevo registro debe no solo encontrar un lugar libre, sino que el lugar debe tener el tamaño suficiente. Para esto, se usan estrategias como "primer ajuste", "mejor ajuste" o "peor ajuste".

Estrategias de Colocación (para reutilización de espacio con registros de longitud variable)

Cuando se eliminan registros de un archivo, especialmente aquellos de longitud variable, el espacio que ocupaban puede ser reutilizado para nuevas inserciones. El desafío es encontrar un espacio "adecuado" que sea lo suficientemente grande para el nuevo registro. Para esto, se utilizan diferentes estrategias de colocación:

→ **Primer Ajuste (First Fit):**

- Definición: Consiste en seleccionar el primer espacio disponible en la lista de espacios libres que sea capaz de almacenar el registro a insertar.
- Funcionamiento: Una vez encontrado, se le asigna el espacio completo al registro (por definición de esta política).
- Ventaja: Minimiza la búsqueda, ya que se detiene en la primera entrada de la lista que cumpla la condición. Es la estrategia más rápida en términos de rendimiento de implantación.
- Desventaja y Fragmentación: No se preocupa por la exactitud del ajuste. Puede generar fragmentación interna porque asigna todo el espacio disponible en ese primer lugar, incluso si el registro no lo ocupa completamente.

→ **Mejor Ajuste (Best Fit):**

- Definición: Consiste en seleccionar el espacio disponible que más se aproxime al tamaño del registro a insertar, es decir, el espacio de menor tamaño donde aún quepa el registro.
- Funcionamiento: Una vez localizado, se le asigna el espacio completo al registro.
- Desventaja y Rendimiento: Exige recorrer toda la lista de espacios disponibles para encontrar el "mejor" ajuste. Por lo tanto, es más ineficiente que el "primer ajuste".
- Fragmentación: Similar al primer ajuste, puede generar fragmentación interna porque se le asigna todo el espacio disponible en el bloque seleccionado, que podría ser mayor al necesario.

→ **Peor Ajuste (Worst Fit):**

- Definición: Consiste en seleccionar el espacio de mayor tamaño disponible en la lista de espacios libres.
- Funcionamiento: A este espacio se le asignan solo los bytes necesarios para el registro, y el resto del espacio libre se mantiene en la lista de disponibles para futuras asignaciones.
- Desventaja y Rendimiento: Requiere recorrer toda la lista de espacios disponibles para encontrar el de mayor tamaño. Es más ineficiente que el "primer ajuste".
- Ventaja y Fragmentación: Es la única técnica que, por definición, no genera fragmentación interna ya que solo asigna el espacio estrictamente necesario.
- Desventaja y Fragmentación: Sin embargo, tiende a generar fragmentación externa, ya que los fragmentos de espacio restantes, aunque disponibles, pueden ser muy pequeños para ser reutilizados eficientemente por nuevos registros. A menudo se requiere un "garbage collector" periódico para recuperar estos espacios no asignados.

Fragmentación

La fragmentación se refiere al desperdicio de espacio en disco. Existen dos tipos principales:

→ **Fragmentación Interna:**

- Definición: Ocurre cuando se asigna más espacio del necesario a un elemento de dato (registro o campo). Este espacio reservado, pero no utilizado, representa un desperdicio.

- Causas típicas:

- Archivos con registros de longitud fija son los que tienden a generar fragmentación interna. Esto sucede porque el tamaño de cada registro está preestablecido por la suma de las longitudes de sus campos, y no siempre se condice con el espacio que el registro realmente utiliza. Por ejemplo, un campo string para un nombre como "Juan Perez" solo usa 10 de los 50 bytes asignados, dejando 40 bytes desperdiciados dentro del registro.

- Las estrategias de "primer ajuste" y "mejor ajuste" para la reutilización de espacio pueden agravarla, ya que asignan el bloque completo encontrado, incluso si es más grande que lo requerido.

- Solución para evitarla: Los registros de longitud variable buscan precisamente evitar este problema al utilizar solo el espacio necesario para almacenar la información.

→ **Fragmentación Externa:**

- Definición: Es el espacio disponible entre dos registros que es demasiado pequeño para poder ser reutilizado. Este espacio no está asignado, pero tampoco está ocupado.

- Causas típicas:

- Es un problema más común en archivos con registros de longitud variable.

- La estrategia de "peor ajuste" puede generarla al dividir un bloque grande y dejar un remanente muy pequeño.

- Impacto: Reduce la eficiencia del uso del disco, ya que hay espacio libre pero no contiguo o suficientemente grande para nuevas asignaciones.

- Solución: Puede requerir algoritmos periódicos de "garbage collection" para consolidar estos pequeños espacios fragmentados.

Bajas

Baja Lógica

La baja **lógica** consiste en marcar un registro como borrado, pero sin quitarlo físicamente del archivo, por lo que sigue ocupando su espacio original.

- **Mecanismo:** Basta con localizar el registro que se desea eliminar y colocarle una marca que indique que no está disponible. Esta operación es eficiente en términos de rendimiento, ya que solo requiere las lecturas necesarias para encontrar el elemento y una única operación de escritura para marcarlo.

- **Ventaja:** Su principal ventaja es la performance, ya que es mucho más rápida que la baja física.

- **Desventaja:** El archivo tiende a crecer continuamente, ya que el espacio de los registros "borrados" no se recupera físicamente.

Recuperación y reasignación de espacio tras una baja lógica

Para reutilizar el espacio liberado lógicamente, se necesitan mecanismos específicos.

- **En archivos de longitud fija:** Cuando se inserta un nuevo registro en un archivo de longitud fija, cualquier espacio disponible es adecuado, ya que todos los registros tienen el mismo tamaño. Para gestionar estos espacios, se puede mantener una lista encadenada de las posiciones borradas en un registro de cabecera. El programa solo necesita consultar esta cabecera para obtener la dirección del primer espacio disponible.

- **En archivos de longitud variable:** A diferencia de los registros de longitud fija, al insertar un nuevo elemento, no basta con que haya un lugar libre, sino que ese espacio debe ser lo suficientemente grande para el nuevo registro. Para gestionar el espacio libre de manera eficiente, se puede generar una lista encadenada invertida de las posiciones borradas, almacenando en un registro de cabecera las direcciones libres y la cantidad de bytes disponibles en cada una para su reutilización.

Existen tres estrategias genéricas para seleccionar el espacio libre para nuevas inserciones:

- **Primer Ajuste (First Fit):** Selecciona el primer espacio disponible en la lista donde quepa el registro. Es el más rápido en rendimiento.

- **Mejor Ajuste (Best Fit):** Busca y selecciona el espacio de menor tamaño disponible que sea lo suficientemente grande para el registro. Requiere recorrer toda la lista de espacios disponibles.

- **Peor Ajuste (Worst Fit):** Busca y selecciona el espacio de mayor tamaño disponible y asigna solo los bytes necesarios para el nuevo registro, dejando el resto del espacio libre. Es menos eficiente en tiempo de ejecución porque debe recorrer toda la lista.

Baja Física

La baja **física** consiste en borrar efectivamente la información del archivo, recuperando el espacio físico que ocupaba.

- **Ventaja:** Permite que el archivo ocupe el mínimo espacio necesario en el disco.

- **Desventaja:** Puede ser costosa en términos de rendimiento, dependiendo de la técnica utilizada y la ubicación del elemento a borrar.

Existen dos técnicas algorítmicas principales para realizar la baja física:

- **Generando un nuevo archivo de datos:** Se crea un archivo nuevo copiando únicamente los elementos válidos del original, omitiendo los que se desean eliminar. Una vez finalizado el proceso, el archivo original se elimina y el nuevo se renombra con el nombre original.

- **Performance:** Requiere leer tantos datos como tenga el archivo original y escribir todos los datos menos el eliminado. Para un archivo de 'n' registros, esto significa 'n' lecturas y 'n-1' escrituras secuenciales.

- **Recursos:** Durante el proceso, se necesita capacidad de almacenamiento suficiente en memoria secundaria para ambos archivos (el original y el nuevo) simultáneamente. Este proceso es también conocido como compactación.

- **Utilizando el mismo archivo de datos:** Se realizan reacomodamientos en el propio archivo para reemplazar el elemento borrado, reduciendo en uno la cantidad de elementos.

- **Mecanismo:** El algoritmo localiza el elemento a borrar, y luego copia sobre este registro el elemento siguiente, repitiendo esta operación hasta el final del archivo. Para asegurar que el archivo se ajuste al nuevo tamaño, se utiliza la instrucción truncate(), que coloca la marca de fin de archivo en la posición indicada por el puntero, eliminando todo lo que hubiera después.

- **Performance:** La cantidad de lecturas es 'n'. La cantidad de escrituras dependerá de la posición del elemento a borrar; en el peor de los casos (si se borra el primer elemento), se deberán realizar 'n-1' escrituras.

- **Recursos:** A diferencia del método anterior, no requiere mayor capacidad en el disco rígido porque se utiliza la ubicación original del archivo.

El proceso de baja (física o lógica) sobre archivos con registros de longitud variable es, a priori, similar a lo discutido para archivos de longitud fija.

Relación con la Fragmentación

La forma en que se manejan las bajas y la reasignación de espacio impacta directamente en la fragmentación del archivo:

Fragmentación Interna

- **Definición:** Ocurre cuando se asigna más espacio a un elemento de dato del que realmente necesita, desperdiciando el espacio no utilizado dentro del registro asignado.

- **Causas principales:**

- Registros de longitud fija: Por definición, un archivo con registros de longitud fija asigna siempre la misma cantidad de bytes por registro, independientemente del contenido real. Por ejemplo, si un campo Nombre se define como string y solo se almacena "Juan Perez" (10 bytes), se desperdician 40 bytes.

- Estrategias de reasignación (en baja lógica de longitud variable): Al reutilizar espacios liberados por baja lógica, las estrategias de Primer Ajuste (First Fit) y Mejor Ajuste (Best Fit) suelen asignar la totalidad del espacio disponible encontrado, incluso si es mayor al requerido por el nuevo registro. Esto genera fragmentación interna.

Fragmentación Externa

- **Definición:** Se refiere al espacio disponible entre registros que es demasiado pequeño para ser reutilizado de manera efectiva por nuevos registros, quedando disperso y no contiguo.

- **Causas principales:**

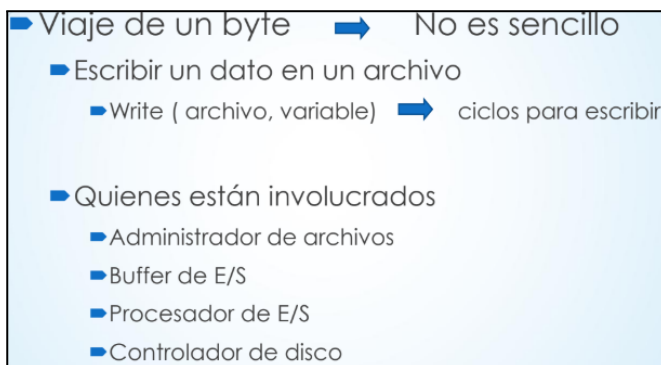
- Registros de longitud variable tras bajas: Cuando se eliminan registros de longitud variable, pueden quedar "huecos" de diferentes tamaños. Si estos huecos son muy pequeños para cualquier nueva inserción, se vuelven inutilizables.

▪ Estrategia de reasignación (en baja lógica de longitud variable): La técnica de Peor Ajuste (Worst Fit), al asignar solo los bytes necesarios del bloque más grande, puede dejar pequeños bloques de espacio libre inutilizable, lo que contribuye a la fragmentación externa. Para recuperar estos espacios, se puede necesitar un algoritmo de "garbage collector" que se ejecute periódicamente.

Organización de datos

Comparado a la memoria RAM, el almacenamiento secundario es más "lento" pero menos costoso para la misma cantidad de bytes. De esta manera, a la hora de recuperar información de una unidad de almacenamiento secundario, se espera que esta venga de una vez o en pocos intentos.

Un **archivo** es una colección de bytes que representa información (ver Definiciones de archivo).



Administrador de archivos

Es un conjunto de programas del S.O. (capas de procedimientos) que tratan aspectos relacionados con archivos y dispositivos de E/S.

- En Capas Superiores: aspectos lógicos de datos (tabla)
 - Establecer si las características del archivo son compatibles con la operación deseada (1).
- En Capas Inferiores: aspectos físicos (FAT)
 - Determinar donde se guarda el dato (cilíndro, superficie, sector) (2).
 - Si el sector está ubicado en RAM se utiliza, caso contrario debe traerse previamente. (3).

Los **buffers de E/S** agilizan la entrada y salida de datos. Manejarlos implica trabajar con grandes grupos de datos en RAM, para reducir el acceso a almacenamiento secundario.

Un **procesador de E/S** es un dispositivo utilizado para la transmisión desde o hacia almacenamiento externo. Independiente de la CPU (3).

Un **controlador de disco** se encarga de controlar las operaciones del disco. Requiere:

- Colocarse en la pista.
- Colocarse en el sector.
- Transferir a disco.

Capas del protocolo de transmisión de un byte

1. El Programa pide al S.O. escribir el contenido de una variable en un archivo.
2. El S.O. transfiere el trabajo al Administrador de archivos.
3. El Adm. busca el archivo en su tabla de archivos y verifica las características.
4. El Adm. obtiene de la FAT la ubicación física del sector del archivo donde se guardará el byte.
5. El Adm se asegura que el sector del archivo está en un buffer y graba el dato donde va dentro del sector en el buffer.
6. El Adm. de archivos da instruccciones al procesador de E/S (donde está el byte en RAM y en que parte del disco deberá almacenarse).
7. El procesador de E/S encuentra el momento para transmitir el dato a disco, la CPU se libera.
8. El procesador de E/S envía el dato al controlador de disco (con la dirección de escritura).
9. El controlador prepara la escritura y transfiere el dato bit por bit en la superficie del disco.

Archivos como secuencia de bytes

- No se puede determinar fácilmente el comienzo y fin de cada dato.
- **Ejemplo: archivos de texto.**

Archivos estructurados

Están dotados de registros y campos, con longitud fija o variable.

Los **campos** son la unidad lógica significativa más pequeña de un archivo, separa la información.

Identidad de campos:

- Longitud predecible o fija: se desperdicia espacio ya que no todos los datos ocupan la misma cantidad de espacio.
- Longitud indicada: se indica al principio de cada campo.
- Delimitador al final de cada campo: carácter especial no usado como dato.

Los **registros** se caracterizan de la siguiente manera:

- Longitud predecible o fija: en cantidad de bytes o cantidad de campos (estos pueden ser de longitud fija o variable también).
- Longitud variable:
 - Indicador de longitud puesto al comienzo.
 - Un segundo archivo que mantiene la información de la dirección del byte de inicio de cada registro.
 - Un delimitador que indica el final.

Búsqueda de información y manejo de índices

Existen estrategias utilizadas para ordenar archivos grandes que no caben completamente en la memoria RAM. Estos métodos buscan mejorar la eficiencia del proceso de ordenamiento al generar particiones iniciales de mayor tamaño antes de realizar la fusión (merge).

Selección por Reemplazo

La selección por reemplazo es un algoritmo de ordenamiento que permite incrementar el tamaño de las particiones de un archivo más allá de la capacidad de la memoria principal disponible. Esto es crucial cuando se manejan archivos de gran tamaño, ya que un menor número de particiones significa menos trabajo durante la etapa de fusión (merge), lo que a su vez reduce los costosos desplazamientos en memoria secundaria.

Funcionamiento

1. Lectura inicial: Se leen desde memoria secundaria tantos registros como quepan en la memoria principal.
2. Inicio de una nueva partición: Se comienza la construcción de una nueva partición ordenada.
3. Selección del menor: De los registros que están actualmente en memoria principal, se selecciona el que tiene la clave menor.
4. Transferencia a secundaria: El registro seleccionado se transfiere a la partición que se está generando en memoria secundaria.
5. Reemplazo y marcaje: El registro elegido en memoria principal es reemplazado por uno nuevo leído desde memoria secundaria. Si la clave de este nuevo registro es menor que la del registro que acaba de ser transferido a secundaria (es decir, ya no mantiene el orden ascendente dentro de la actual partición), este nuevo registro se marca como "no disponible" o "dormido" (entre paréntesis en el ejemplo) y formará parte de la siguiente partición.
6. Repetición: Los pasos 3 a 5 se repiten hasta que todos los registros en la memoria principal estén marcados como "no disponibles".
7. Nueva partición: Se inicia una nueva partición, activando todos los registros "no disponibles" en memoria principal, y se repite el proceso desde el paso 3 hasta agotar los elementos del archivo original.

Ventajas

- Este método aumenta el tamaño de las particiones, en promedio, al doble de la cantidad de registros que caben en la memoria principal (2P).
- Si los datos de entrada están parcialmente ordenados, el tamaño promedio de las particiones puede ser incluso superior al doble (2P).

Desventajas

- Tiende a generar muchos registros "no disponibles", lo que implica un manejo adicional.

- Las particiones resultantes no son necesariamente de igual tamaño, lo cual puede afectar la eficiencia de algunas variantes del proceso de fusión posterior.

Ejemplo Ilustrativo (Ejemplo 5.2 del libro)

Imaginemos las siguientes claves en memoria secundaria: 34, 19, 25, 59, 15, 18, 8, 22, 68, 13, 6, 48, 17 (17 es el principio de la cadena de entrada). Supongamos que la memoria principal admite solo 3 claves.

Inicio de la Primera Partición:

Resto de la entrada	Mem. principal	Partición generada
34, 19, 25, 59, 15, 18, 8, 22, 68, 13	6 48 17	–
34, 19, 25, 59, 15, 18, 8, 22, 68	13 48 17	6
34, 19, 25, 59, 15, 18, 8, 22	68 48 17	13, 6
34, 19, 25, 59, 15, 18, 8	68 48 22	17, 13, 6
34, 19, 25, 59, 15, 18	68 48 (8)	22, 17, 13, 6
34, 19, 25, 59, 15	68 (18) (8)	48 , 22, 17, 13, 6
34, 19, 25, 59	(15) (18) (8)	68, 48, 22, 17, 13, 6

Primera partición completa; se inicia la construcción de la siguiente:

Resto de la entrada	Mem. principal	Partición generada
34, 19, 25, 59	(15) (18) (8)	–
34, 19, 25	15 18 59	8
34, 19	25 18 59	15, 8
34	25 19 59	18, 15, 8
	25 34 59	19, 18, 15, 8
	– 34 59	25, 19, 18, 15, 8
	– – 59	34, 25, 19, 18, 15, 8
	– – –	59, 34, 25, 19, 18, 15, 8

Selección Natural

La selección natural es una variante del método de selección por reemplazo. La principal diferencia es que la selección natural reserva y utiliza memoria secundaria para insertar los registros que se marcan como "no disponibles" o "dormidos". La generación de una partición finaliza cuando este espacio secundario reservado para los registros "no disponibles" se llena (entra en overflow).

Ventajas

- Al igual que la selección por reemplazo, produce particiones con un tamaño promedio igual o mayor al doble de la cantidad de registros que caben en la memoria principal.

Desventajas

- También tiende a generar muchos registros "no disponibles".

- Las particiones resultantes no quedan necesariamente de igual tamaño.

En el contexto de la algoritmia clásica de archivos, estos métodos son una mejora significativa sobre el "sort interno" tradicional, que es muy costoso para archivos grandes ya que implica leer y escribir cada elemento muchas veces directamente en memoria secundaria. Al generar particiones más grandes en la memoria principal y luego escribirlas secuencialmente, se reducen los costosos accesos y desplazamientos del cabezal de lectura/escritura del disco. Sin embargo, a diferencia de otras estrategias de administración de espacio como "primer ajuste", "mejor ajuste" o "peor ajuste" que se aplican a la reutilización de espacios vacíos por eliminaciones dentro de archivos, la selección por reemplazo y natural se centran en la creación de particiones ordenadas para un proceso de ordenamiento por fusión.

Indización

Un **índice** es una estructura de datos adicional diseñada para agilizar el acceso a la información almacenada en un archivo. Funciona como una tabla que asocia una clave de búsqueda con una referencia a la ubicación del registro correspondiente dentro del archivo de datos. Una de sus características fundamentales es que permite imponer un orden lógico en un archivo sin necesidad de reacomodar físicamente los datos.

Propósito y Funcionamiento Principal

El objetivo principal de un índice es minimizar el número de accesos a memoria secundaria (disco) necesarios para encontrar información, ya que los accesos a disco son considerablemente más costosos en tiempo que las operaciones en memoria principal.

Cuando se busca un dato en un archivo indizado, el proceso generalmente implica dos pasos:

1. Búsqueda en el índice: Se localiza la clave deseada en la estructura del índice. Dado que el índice suele ser un archivo de menor tamaño y con registros de longitud fija, es posible realizar una búsqueda binaria sobre él, lo que es mucho más eficiente que una búsqueda secuencial en el archivo de datos original. Si el índice cabe en la memoria principal, la búsqueda es aún más rápida.

2. Acceso directo al archivo de datos: Una vez que se encuentra la clave en el índice, se obtiene la dirección física o el Número Relativo de Registro (NRR) del registro completo en el archivo de datos. Con esta referencia, se accede directamente al registro deseado en el archivo de datos.

Estructura y Tipos de Índices

Generalmente, un índice es otro archivo, independiente del archivo de datos original. Este archivo de índice contiene pares de (clave, dirección) y está organizado con registros de longitud fija.

Los tipos de índices se clasifican principalmente según el tipo de clave que referencian:

Índice Primario

Se crea a partir de la clave primaria del archivo de datos. Una clave primaria no admite valores repetidos y, en general, no se modifica.

- **Ventajas:** Al ser más pequeño que el archivo de datos y tener registros de longitud fija, permite una mejor performance de búsqueda (por ejemplo, con búsqueda binaria). También facilita el uso de estructuras de datos más eficientes para su organización, como los árboles balanceados.
- **Operaciones:**

- Creación: Se crea vacío, solo con un registro de encabezado, al mismo tiempo que el archivo de datos.
- Altas: Al agregar un nuevo registro al archivo de datos, su clave primaria y dirección se insertan de forma ordenada en el índice.
- Modificaciones: Si la clave primaria no cambia, y el registro no se reubica físicamente, el índice no se altera. Si el registro se agranda y se reubica, la nueva posición debe actualizarse en el índice. Si la clave primaria cambia, la operación se considera una eliminación y una nueva alta en el índice.
- Bajas: Al eliminar un registro del archivo de datos, la información asociada en el índice primario también se elimina, ya sea física o lógicamente.

Índices para Claves Candidatas

Las claves candidatas son similares a las claves primarias en que no permiten valores repetidos, pero no fueron seleccionadas como clave primaria principal.

El tratamiento y la gestión de los índices para claves candidatas son similares a los de los índices primarios.

Índices Secundarios

Permiten buscar información utilizando claves secundarias, que son atributos que pueden contener valores repetidos (ej. nombre de canción, autor) y son más fáciles de recordar que una clave primaria.

Funcionan relacionando una clave secundaria con una o más claves primarias. El flujo de acceso es: búsqueda en índice secundario por clave secundaria, que retorna la clave primaria; luego, uso de la clave primaria en el índice primario para obtener la dirección física del registro en el archivo de datos. Esta estructura en cascada (secundario -> primario -> dato) protege los índices secundarios de cambios en la ubicación física de los registros de datos, ya que solo el índice primario necesita actualizar su referencia física.

Organización Alternativa (Listas Invertidas): En lugar de repetir la clave secundaria para cada ocurrencia, se puede almacenar en un único registro del índice la clave secundaria junto con una lista o arreglo de las claves primarias asociadas. Esto optimiza el espacio y reduce el costo de reacomodo al agregar nuevas ocurrencias.

Índices Selectivos

Contienen solo claves asociadas a una parte específica de la información que es de mayor interés para consultas frecuentes. Actúan como un filtro, mejorando el acceso a un subconjunto particular de datos.

Performance en acceso secuencial

- Mejor caso: leer 1 registro.
- Peor caso: leer N registros.
- Promedio: $N/2$ comparaciones (lecturas).
- De $O(N)$.

Performance en acceso directo

- Mejor y peor caso: leer 1 registro.
- De $O(1)$.
- Se requiere saber dónde acceder.
- Aplicable a registros de longitud fija.

El acceso directo es preferible sólo cuando se necesitan pocos registros específicos, pero este método NO siempre es el más apropiado para la extracción de información.

Por ejemplo: generar cheques de pago a partir de un archivo de registros de empleados. Como todos los registros se deben procesar es más rápido y sencillo leer registro a registro desde el principio hasta el final, y NO calcular la posición en cada caso para acceder directamente.

Número relativo de registro (NRR)

Indica la posición relativa con respecto al principio del archivo.

- Solo aplicable con registros de longitud fija.
- Por ejemplo: NRR 546 y longitud de cada registro 128 bytes distancia en bytes= $546 * 128 = 69.888$.

2) Árboles

Árbol binario

Es una estructura de datos donde cada nodo tiene a lo sumo dos hijos. En memoria se organizan por un índice $0 \leq i \leq n-1$ donde n es la cantidad de nodos del árbol. Cada referencia a dónde apunta el índice contiene la clave del nodo, y las referencias a sus hijos izq. y der:

Raíz → 0			
	Clave	Hijo izq	Hijo Der
0	MM	1	2
1	GT	3	4
2	ST	8	11
3	BC	5	6
4	JF	7	14
5	AB	-1	-1
6	CD	-1	-1
7	HI	-1	-1

	Clave	Hijo izq	Hijo Der
8	PR	9	10
9	OP	-1	-1
10	RX	-1	-1
11	UV	12	13
12	TR	-1	-1
13	ZR	-1	-1
14	KL	-1	-1

Árbol AVL

Árbol balanceado: un árbol está balanceado cuando la altura de la trayectoria más corta hacia una hoja no difiere de la altura de la trayectoria más grande.

Un **árbol AVL** es un árbol binario balanceado en altura (BA(1)) en el que las inserciones y eliminaciones se efectúan con un mínimo de accesos. 1 es el nivel de balance, es decir, que la mayor diferencia que puede haber entre las alturas de cualesquiera nodos diferentes **es 1**.

En la performance del peor caso posible:

Binario: Búsqueda: $\log_2(N+1)$

AVL: Búsqueda: $1.44 * \log_2(N+2)$

Paginación de árboles binarios

La paginación de árboles binarios es una estrategia fundamental en la organización de datos para mejorar la eficiencia del acceso a la información almacenada en memoria secundaria, como los discos rígidos.

Un árbol binario paginado consiste en dividir un árbol binario en unidades llamadas "páginas", donde cada página contiene un conjunto de nodos del árbol. Estos nodos están ubicados en direcciones físicas cercanas para optimizar la transferencia de datos.

Se basa en el concepto de buffering, donde la transferencia de datos desde o hacia el disco rígido se realiza en bloques (buffers o páginas completas) en lugar de un solo registro individual. Esto minimiza el número de accesos al disco.

Propósito y Ventajas (Minimización de Accesos a Disco)

- El objetivo principal es reducir drásticamente la cantidad de accesos a disco necesarios para recuperar información. Al transferir una página completa, se recuperan varios nodos del árbol con un solo acceso, en lugar de un acceso por cada nodo.
- Por ejemplo, si una página puede contener siete nodos, con dos accesos a disco sería posible recuperar un nodo específico entre 63, asumiendo que las direcciones físicas son contiguas y los desplazamientos son despreciables.
- La performance de búsqueda se mejora significativamente. Si un buffer puede contener K elementos, la eficiencia de búsqueda se transforma de ser de orden $\log_2(N)$ a $\log_{K+1}(N)$, donde N es la cantidad total de claves y K es la cantidad de nodos por página. Esto representa una mejora sustancial en comparación con la búsqueda binaria tradicional sobre un archivo.

Desafíos y Limitaciones

- La construcción de un árbol binario paginado es compleja. Para que las páginas sean eficientes, los elementos que las componen deberían llegar consecutivamente para ser almacenados en el mismo sector del disco. Sin embargo, no es posible condicionar la llegada de los elementos, lo que puede resultar en páginas incompletas o la necesidad de que cada elemento se almacene en la página que le corresponde lógicamente.

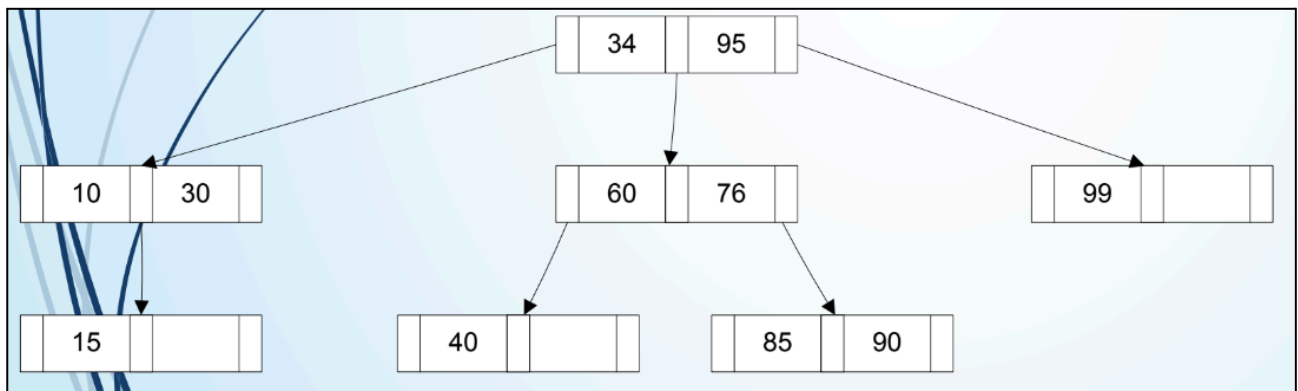
◦ La división del árbol en páginas implica un costo adicional para reacomodarlo y mantener su balanceo interno. Un algoritmo que soporte esta construcción sería muy costoso de implementar en términos de performance.

◦ Aunque la paginación mejora la eficiencia de acceso a disco, no resuelve el problema fundamental de los árboles binarios, que tienden a desbalancearse fácilmente. Un árbol desbalanceado puede transformar la búsqueda logarítmica en una búsqueda lineal, disminuyendo drásticamente la performance.

◦ Debido a estas limitaciones, la solución a este problema reside en la adopción de estructuras más avanzadas que manipulan múltiples registros como una unidad (páginas) y que están diseñadas para mantener un balanceo a bajo costo.

Árbol multiamino

Un **árbol multiamino** es una estructura de datos en la cual cada nodo puede contener k elementos y $k+1$ hijos. El **orden** de un árbol multiamino es la máxima cantidad de descendientes posibles de un nodo.



Árbol B

Es un árbol multiamino balanceado con una construcción especial en forma ascendente que permite mantenerlo balanceado a bajo costo.

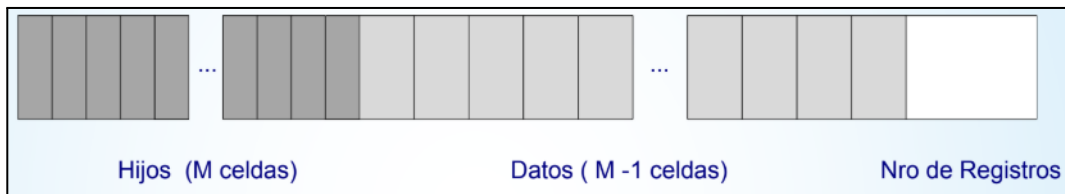
Propiedades

(Orden = M)

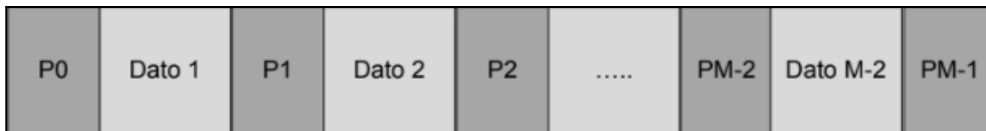
- Ningún nodo tiene más de M hijos
- $C/nodo$ (menos raíz y los terminales) tienen como mínimo $\lceil M/2 \rceil$ hijos
- La raíz tiene como mínimo 2 hijos (o sino ninguno)
- Todos los nodos terminales a igual nivel
- Nodos no terminales con K hijos contienen $K-1$ registros. Los nodos terminales tienen:
- Mínimo $\lceil M/2 \rceil - 1$ registros
- Máximo $M - 1$ registros

Cada nodo de un árbol B se puede representar de la siguiente manera:

- Formato del nodo:

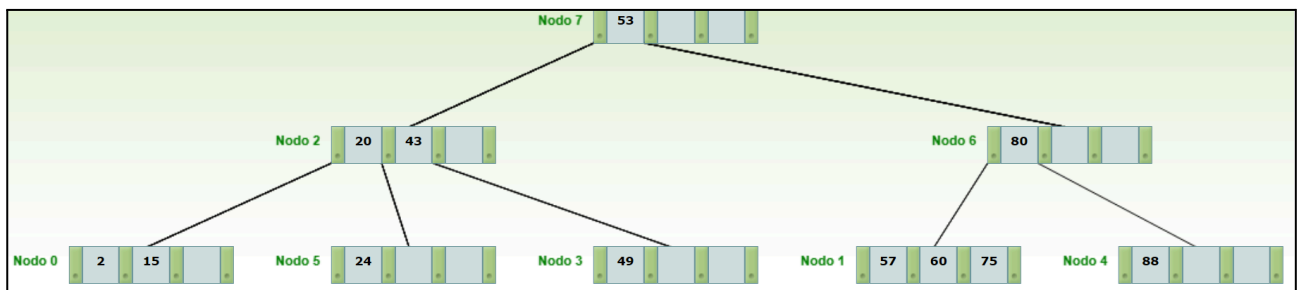


- Formato gráfico del nodo:



Ejemplo:

- Árbol resultante de agregar 12 claves (43, 2, 53, 88, 75, 80, 15, 49, 60, 20, 57, 24):



- Historial de operaciones (de más reciente a más vieja):
 - División de la raíz, aumento de altura
 - Desborde del nodo 0. División y promoción del elemento 20
 - Inserción elemento 57 en el nodo 1
 - Inserción elemento 20 en el nodo 0
 - Desborde del nodo 1. División y promoción del elemento 80
 - Desborde del nodo 0. División y promoción del elemento 43
 - Inserción elemento 15 en el nodo 0
 - Inserción elemento 80 en el nodo 1
 - Inserción elemento 75 en el nodo 1
 - División de la raíz, aumento de altura
 - Inserción elemento 53 en el nodo 0
 - Inserción elemento 2 en el nodo 0
 - Inserción elemento 43 en el nodo 0

- Se creo el árbol B con un orden 4

Eficiencia

Siendo H la altura del árbol...

Operación	Tipo de Caso	Lecturas	Escrituras
Búsqueda	Mejor Caso	1	0
	Peor Caso	H	0
Inserción	Mejor Caso	H	1
	Peor Caso	H	$(2 * H) + 1$
Eliminación	Mejor Caso	Borra un elemento del nodo y no produce underflow, solo reacomodos (# elementos $\geq [M/2]-1$). Entonces, serán H lecturas.	1
	Peor Caso	Se produce underflow (#elementos $< [M/2] - 1$). Entonces, serán $(2 * H) - 1$ lecturas.	$H - 1$
Modificación	Caso General (equivalente a Baja + Alta)	$(H + (2H-1))$	$(1 + (H-1))$

Varios estudios empíricos han determinado que, en un árbol B de orden $M = 10$, habrán aprox. un 25% de overflows. Mientras que, en un orden de $M = 100$, este porcentaje se reduce a 2 puntos. Analizando esta última situación, 98 de cada 100 inserciones se tratan por el mejor caso, en tanto que solamente dos no están dentro de esta consideración, lo que no significa necesariamente que se trate del peor caso.

Por lo tanto, la inserción de nuevos elementos en el árbol B requiere un número considerablemente bajo de operaciones de entrada-salida para lograr mantener el orden en la estructura. Queda aún analizar el proceso de baja y su eficiencia

Árbol B+

Es otro tipo de árbol multicamino con las siguientes propiedades:

- Cada nodo del árbol puede contener, a lo máximo, M (n° de orden) descendientes y $M-1$ elementos.
- La raíz no posee descendientes o tiene al menos dos.
- Un nodo con x descendientes contiene $x-1$ elementos.
- Los nodos terminales tienen, como mínimo, $(\lceil M/2 \rceil - 1)$ elementos, y como máximo, $M-1$ elementos.
- Los nodos que no son terminales ni raíz tienen, como mínimo, $\lceil M/2 \rceil$ descendientes.
- Todos los nodos terminales se encuentran al mismo nivel.
- Los nodos terminales representan un conjunto de datos y están enlazados entre ellos.

Si se compara la definición anterior con la resultante para árboles B, se encontrarán similitudes, excepto en la última propiedad.

Es esta última propiedad la que establece la principal diferencia entre un árbol B y un árbol B+. Para poder realizar acceso secuencial ordenado a todos los registros del archivo, es necesario que cada elemento (clave asociada a un registro de datos) aparezca almacenado en un nodo terminal. Así, los árboles B+ diferencian los elementos que constituyen datos de aquellos que son separadores.

Árbol B+ de prefijos simples

Un **árbol B+ de prefijos simples** es una variación avanzada del árbol B+ que tiene como objetivo principal optimizar el uso del espacio físico en el disco. Para comprender esta estructura, es útil primero recordar las características fundamentales de un árbol B+.

Árboles B+ (Balanceados)

Un árbol B+ es una estructura de datos multicamino con propiedades específicas que lo hacen muy eficiente para la gestión de índices en bases de datos. Las propiedades clave de un árbol B+ incluyen:

- Cada nodo del árbol puede contener, como máximo, M descendientes y $M-1$ elementos.
- La raíz, si no tiene descendientes directos, tiene al menos dos.
- Un nodo con x descendientes contiene $x-1$ elementos.
- Todos los elementos de datos (claves asociadas a un registro) se almacenan exclusivamente en los nodos terminales (u hojas). Esto contrasta con los árboles B, donde los datos pueden estar en cualquier nodo.
- Los nodos terminales están enlazados entre sí, formando un conjunto de datos que permite un acceso secuencial eficiente a todos los registros del archivo en orden.

- Los nodos no terminales, incluyendo la raíz, actúan como separadores y contienen una copia de los elementos presentes en los nodos terminales para guiar la búsqueda.

Durante el proceso de creación de un árbol B+, cuando un nodo se satura, se produce una división. Una copia de la menor de las claves mayores de la segunda mitad de los elementos se promociona al nodo padre para actuar como separador.

Árboles B+ de Prefijos Simples

La innovación de los árboles B+ de prefijos simples se centra en cómo se manejan estos separadores en los nodos no terminales para optimizar aún más el espacio. En lugar de promocionar la clave completa como separador, esta variante utiliza la mínima expresión posible de la clave que sea suficiente para decidir la ruta de búsqueda (hacia la izquierda o hacia la derecha) en el árbol.

Esta estrategia permite que los separadores sean más cortos, lo que significa que más referencias pueden caber en cada nodo no terminal. Al aumentar la cantidad de referencias por nodo, se puede reducir la altura total del árbol. Una menor altura del árbol implica una menor cantidad de accesos a disco necesarios para encontrar un elemento, lo que mejora la eficiencia de las búsquedas.

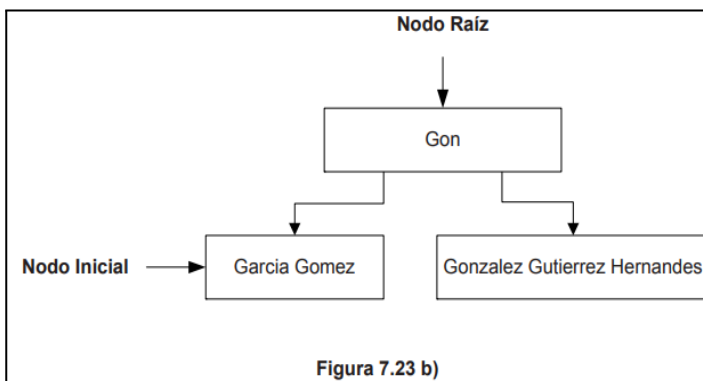
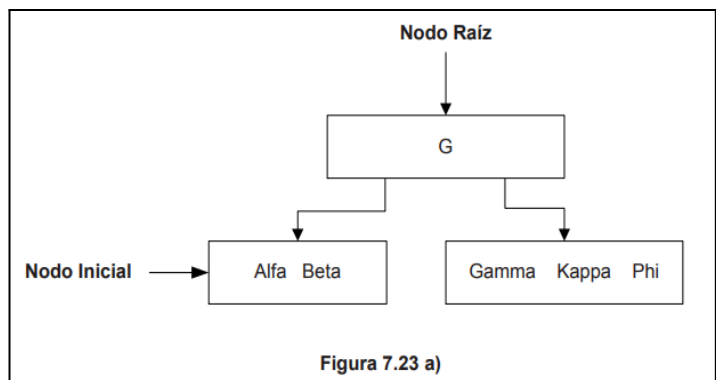
Ejemplos para Ilustrar la Minimalidad del Prefijo:

El libro proporciona ejemplos claros para entender cómo funcionan estos separadores de prefijos simples:

1. Ejemplo 1 (Figura 7.23 a):

- Si al dividir un nodo terminal que contiene, por ejemplo, las claves "Alfa", "Beta", "Gamma", "Kappa", "Phi", la clave "Gamma" sería la que se promocionaría como separador en un árbol B+ tradicional.

- Sin embargo, en un árbol B+ de prefijos simples, si la letra "G" es suficiente para diferenciar el rango de claves que comienza con "Gamma" de las claves que le preceden (por ejemplo, "Alfa" y "Beta"), entonces solo la letra "G" se utiliza como separador en el nodo padre.



2. Ejemplo 2 (Figura 7.23 b):

- Considere un escenario donde las claves que se deben separar son "Garcia", "Gomez", "Gonzalez", "Gutierrez", "Hernandez".

- En este caso, si solo se promocionara la letra "G", no sería suficiente para diferenciar entre "Garcia", "Gomez", "Gonzalez", etc., ya que todas comienzan con "G".

Por lo tanto, el separador debe ser más largo. El libro indica que el prefijo "Gon" podría ser la "mínima expresión posible" para diferenciar las

claves que comienzan con "Gon" (como "Gonzalez") de las que comienzan con otros prefijos ("Gar", "Gom") o de las que le siguen alfabéticamente ("Gutierrez", "Hernandez").

Eficiencia

Siendo H la altura del árbol...

Operación	Tipo de Caso	Lecturas (accesos a disco)	Escrituras (accesos a disco)	Comparación con Árbol B	Notas
Búsqueda	Mejor Caso	1	0	Igual	
	Peor Caso	H	0	Igual	
Inserción	Mejor Caso	H	1	Similar	Puede requerir + tiempo
	Peor Caso	H	$(2 * H) + 1$	Similar	Puede requerir + tiempo
Eliminación	Mejor Caso	H	1	Similar	Puede requerir + tiempo
	Peor Caso	$(2 * H) - 1$	H - 1	Similar	Puede requerir + tiempo
Acceso Secuencial	N/A	Rápido (con punteros)	N/A	Mejor (B es Lento/Complejo)	Principal ventaja de B+

Árbol B*

Un árbol B* es un árbol balanceado con las siguientes propiedades especiales:

- Cada nodo del árbol puede contener, como máximo, M descendientes y M-1 elementos.
- La raíz no posee descendientes o tiene al menos dos.
- Un nodo con x descendientes contiene x-1 elementos.
- Los nodos terminales tienen, como mínimo, $\lceil (2M-1)/3 \rceil - 1$ elementos, y como máximo, M-1 elementos.
- Los nodos que no son terminales ni raíz tienen, como mínimo, $\lceil (2M-1) / 3 \rceil$ descendientes.
- Todos los nodos terminales se encuentran al mismo nivel.

La búsqueda y la eliminación de claves son tratadas casi de la misma manera que en los árboles B.

La única excepción a las propiedades de capacidad mínima ocurre cuando la raíz, siendo el único nodo, se completa y se divide, generando dos hijos que solo llenarán su espacio a la mitad.

Inserción y su eficiencia

El proceso de inserción en un árbol B* puede ser regulado de acuerdo con tres políticas básicas: política de un lado, política de un lado u otro lado, política de un lado y otro lado.

Así, cada política determina, en caso de overflow, el nodo adyacente hermano a tener en cuenta. La política de un lado determina que el nodo hermano adyacente considerado será uno solo, definiendo la política de izquierda, o en su defecto, la política de derecha. En caso de completar un nodo, intenta redistribuir con el hermano indicado. En caso de no ser posible porque el hermano también está completo, tanto el nodo que genera overflow como dicho hermano son divididos de dos nodos llenos a tres nodos dos tercios llenos.

La inserción en un árbol B* se discute con mayor detalle debido a las diferentes políticas de manejo de overflow. A diferencia de los árboles B, que ante un desborde directamente dividen el nodo, los árboles B* primero intentan una redistribución con un nodo adyacente hermano antes de proceder a una división. Esta estrategia pospone la creación de nuevas páginas, haciendo que el árbol crezca en altura más lentamente y sea más eficiente en el uso del espacio.

Las políticas de inserción en árboles B* ante un overflow incluyen:

Política de un lado (ej. "Derecha" o "Izquierda"):

- Si un nodo se completa, intenta redistribuir con un hermano adyacente (por ejemplo, el de la derecha).
- Si el hermano también está completo, se dividen los dos nodos llenos en tres nodos que estarán llenos en dos tercios (2/3).
- Costos de acceso a disco (en caso de overflow):
 - Mejor caso (redistribución posible): 2 lecturas (el nodo que se satura y un adyacente hermano) y 2 escrituras (para actualizar el nodo y su hermano).
 - Peor caso (se requiere división): 2 lecturas y 3 escrituras (el nodo padre, los dos nodos que estaban completos y el nuevo que se genera).

Política de un lado u otro lado (ej. "Izquierda o Derecha"):

- En caso de saturación, se intenta primero redistribuir con un hermano adyacente. Si no es posible, se intenta con el otro hermano adyacente.
- Si ninguna redistribución es posible, la alternativa es dividir dos nodos llenos en tres nodos con dos tercios de ocupación.
- Costos de acceso a disco (en caso de overflow):
 - Mejor caso (redistribución posible): 2 lecturas y 2 escrituras.
 - Peor caso (se requiere división): 3 lecturas y 3 escrituras.

Política de ambos adyacentes hermanos (ej. "Izquierda y Derecha"):

- Similar al caso anterior, pero si los tres nodos (el saturado y ambos hermanos adyacentes) están completos, se toman los tres nodos y se generan cuatro nodos con tres cuartas partes ($3/4$) de ocupación cada uno. Esta política genera árboles de menor altura, aunque puede requerir un mayor número de operaciones de entrada-salida sobre disco para su implementación.
- Costos de acceso a disco (en caso de overflow):
 - Mejor caso (redistribución posible): 2 lecturas y 2 escrituras.
 - Peor caso (se requiere división): 3 lecturas y 4 escrituras.

En general, las operaciones de inserción en árboles B^* son un poco más lentas que en los árboles B debido a la complejidad adicional de la redistribución y las divisiones en dos tercios o tres cuartos, pero esta complejidad se justifica por la mejora en la utilización del espacio y la menor altura resultante del árbol.

Eliminación y su eficiencia

La operación de eliminación de un elemento en un árbol B^* es similar a la de los árboles B . Siempre se busca eliminar el elemento de un nodo terminal. Si el elemento a eliminar no está en un nodo terminal, se intercambia con el menor de sus claves mayores (residente en un nodo terminal) para luego eliminarlo.

Al igual que en los árboles B , si la eliminación provoca un underflow (el nodo tiene menos del mínimo de elementos), se intenta una redistribución con un nodo adyacente hermano. Si la redistribución no es posible, se procede a una concatenación de nodos. La concatenación puede propagar cambios hacia la raíz y, en el peor de los casos, disminuir la altura del árbol.

Los costos de acceso a disco para la eliminación son los mismos que para los árboles B :

- Mejor caso (sin underflow): H lecturas y 1 escritura.
- Peor caso (con concatenación que se propaga): $(2 * H) - 1$ lecturas y $H - 1$ escrituras.

Conclusión sobre la Eficiencia de los Árboles B^*

Los árboles B^* mejoran la eficiencia de los árboles B principalmente al:

- Aumentar la ocupación mínima de los nodos: Aseguran que los nodos estén llenos en al menos 2/3 partes, lo que reduce la cantidad de nodos totales y, por ende, la altura del árbol para un mismo volumen de datos. Una menor altura significa menos accesos a disco en las operaciones de búsqueda.
- Posponer el crecimiento de la altura: La estrategia de redistribución antes de la división en la inserción ayuda a evitar el aumento prematuro de niveles, lo que contribuye a un mejor rendimiento general.

Sin embargo, el costo de estas mejoras se manifiesta en que las operaciones de inserción pueden ser un poco más lentas debido a la mayor complejidad algorítmica de la redistribución y las divisiones. A pesar de esto, los árboles B* siguen siendo una solución muy eficiente para la administración de índices, especialmente en entornos de bases de datos que requieren búsquedas rápidas.

Buffers y árboles B virtuales

Los árboles B virtuales están intrínsecamente ligados al manejo de buffers. Cuando se transfiere información desde o hacia el disco rígido, esta transferencia no se limita a un solo registro, sino que se envía un conjunto de registros, es decir, aquellos que caben en un buffer de memoria. Un buffer es una memoria intermedia (ubicada en la memoria RAM) donde los datos residen provisionalmente antes de ser almacenados definitivamente en memoria secundaria o después de ser recuperados de ella. Esta forma de trabajo optimiza la performance de lectura y escritura sobre archivos.

La eficiencia de las operaciones en los árboles B depende directamente de la altura del árbol (H), la cual se busca mantener acotada. Sin embargo, el hecho de que un árbol B tenga H niveles no significa que se necesiten exactamente H accesos a disco para recuperar un elemento. Aquí es donde el manejo de buffers juega un papel crucial.

Un Sistema Operativo (SO) administra un número significativo de buffers. Según la política de trabajo del SO, es posible que los buffers que contienen los nodos más utilizados se mantengan en memoria principal (RAM), con el objetivo de minimizar el número de accesos a disco.

LRU (Last Recently Used)

Una de las políticas más utilizadas para el manejo de buffers es LRU (Last Recently Used). Bajo esta política, los nodos que han sido visitados más recientemente se mantienen en memoria principal.

Impacto en la Eficiencia de los Árboles B

Cuando un árbol B administra un índice que se accede con mucha frecuencia, el SO tiende a mantener el buffer que contiene el nodo raíz en memoria principal debido a su uso constante. Esto tiene un efecto directo en la eficiencia: disminuye en uno el número de accesos a disco necesarios para recuperar la información.

Empíricamente, si se tiene un árbol de tres niveles:

- La primera búsqueda de un elemento podría requerir hasta tres accesos a disco.
- La segunda búsqueda requeriría como máximo dos accesos a disco, ya que la raíz ya se mantendría en memoria principal.

- Buscar cinco elementos podría requerir en promedio 1.71 accesos a disco, demostrando que la cantidad de accesos necesarios disminuye a medida que se intenta localizar más información.

Esta mejora empírica resalta una característica adicional que hace a los árboles B (y por extensión a los B*) aún más eficientes en su utilización. Al mantener las páginas más utilizadas en RAM, los "árboles B virtuales" optimizan la interacción con la memoria secundaria, reduciendo el costo real de las operaciones a pesar de la complejidad logarítmica de las búsquedas.

Conclusiones

La familia de los árboles balanceados son estructuras muy poderosas y flexibles para la administración de índices asociados a archivos de datos, con un nivel de performance muy interesante. Sin embargo, no deben considerarse como la única solución posible a todos los problemas.

La elección de la estructura de un archivo depende de su naturaleza y finalidad. Por ejemplo, si un archivo de datos es pequeño y sus registros caben en un solo nodo, usar un árbol es innecesario, ya que añade un nivel de indirección por cada índice y no es conveniente si las consultas son infrecuentes.

Existen otras metodologías de organización de archivos que ofrecen mejores tiempos de respuesta para ciertas necesidades de rendimiento, por lo que los árboles balanceados no son la única solución óptima.

Aun así, la familia de árboles B es muy útil cuando se requiere tanto una búsqueda aleatoria eficiente como un acceso secuencial al archivo. Las características comunes de los árboles de la familia B incluyen:

- **Manejo de bloques o nodos de trabajo:** Esto optimiza las operaciones de entrada/salida y limita el número de accesos necesarios.
- **Balanceo:** Todos los nodos terminales se encuentran a la misma distancia de la raíz.
- **Crecimiento:** A diferencia de los árboles binarios, crecen de abajo hacia arriba.
- **Forma:** Son "bajos" y "anchos", en contraste con los árboles binarios que son "altos" y "delgados".
- **Versatilidad:** Permiten implementar algoritmos que manejan registros de longitud variable, mejorando el rendimiento y el uso del espacio.

Específicamente, los árboles B* superan a los árboles B en eficiencia al incrementar la cantidad mínima de elementos por nodo, lo que reduce la altura del árbol. Sin embargo, esto hace que las inserciones sean un poco más lentas.

En cuanto a los árboles B+, toda la información de las claves se guarda en los nodos terminales. Los nodos no terminales funcionan como separadores y contienen una copia de los elementos presentes en los nodos terminales.

3) Dispersión

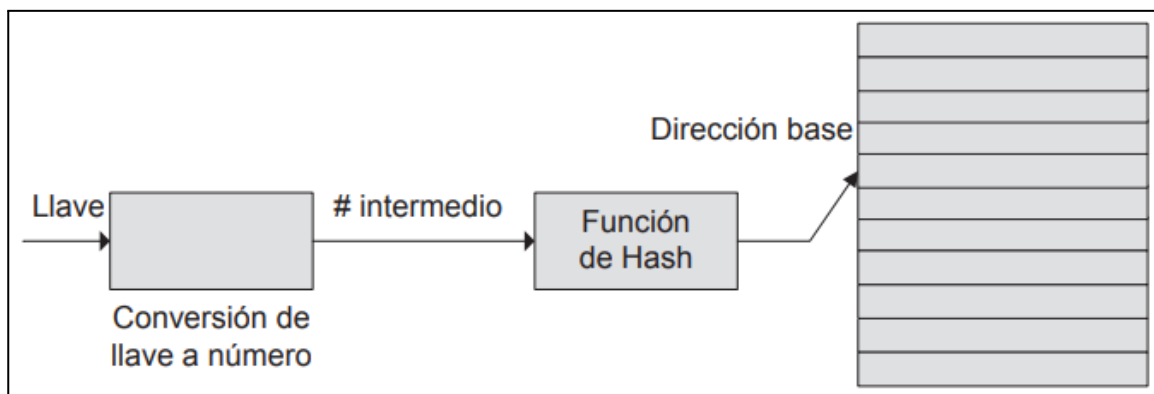
Definiciones

1. Técnica para generar una dirección base única para una llave dada. La dispersión se usa cuando se requiere acceso rápido a una llave.
2. Técnica que convierte la llave del registro en un número aleatorio, el que sirve después para determinar dónde se almacena el registro.
3. Técnica de almacenamiento y recuperación que usa una función de hash para mapear registros en dirección de almacenamiento.

La **dispersión de archivos** sirve para generar una dirección base única para una clave dada:

- Convierte la clave en un número aleatorio, que luego sirve para determinar dónde se almacena la clave.
- Utiliza una función de dispersión para mapear cada clave con una dirección física de almacenamiento.
- Utilizada cuando se requiere acceso rápido por clave.

Aquí puede observarse cómo la llave primero se convierte en un número, sobre el cual puede aplicarse la función de hash, que permite obtener la dirección donde el registro se almacenará dentro del archivo:



Características

Costos

- No podemos usar registros de longitud variable
- No puede haber orden físico de datos
- No permite llaves duplicadas

Direccionamiento estático

En este tipo de dispersión, el espacio disponible para dispersar los registros del archivo está fijado previamente.

Direccionamiento dinámico

El espacio disponible para dispersar los registros del archivo aumenta o disminuye en función de las necesidades.

Función de dispersión

Es una “caja negra” que, a partir de una clave, genera la dirección donde se va a almacenar el registro → función de hash.

Densidad de empaquetamiento

La Densidad de Empaquetamiento (DE) es un parámetro fundamental en el método de dispersión (hashing) para la organización de archivos. Su propósito principal es evaluar la eficiencia de cómo se utiliza el espacio de almacenamiento y predecir la ocurrencia de colisiones y desbordes (overflow) en un archivo disperso.

Definición y Cálculo

- La DE se define como la razón entre la cantidad de registros (r) que componen un archivo y el espacio total disponible para almacenar ese archivo.
- El espacio disponible se calcula multiplicando el número de nodos direccionables (n) por la función de hash y la cantidad de registros que cada nodo puede almacenar (Registros por nodo, RPN).
- **La fórmula es:** $DE = r / (RPN * n)$
- Por ejemplo, si se dispersan 30 registros en 10 direcciones, y cada dirección tiene capacidad para 5 registros, la DE es 0.6 o 60% ($30 / (5 * 10)$).

Impacto en la Performance y el Espacio

- Baja DE: Implica desperdicio de espacio en el disco, ya que se reserva más lugar del que realmente se utiliza, generando fragmentación. Sin embargo, a cambio, reduce significativamente la probabilidad de overflow, lo que mejora la eficiencia en la búsqueda, ya que menos registros necesitarán reubicarse por colisiones.
- Alta DE: Optimiza el uso del espacio en disco, pero a costa de un mayor porcentaje de colisiones y saturación, lo que puede aumentar el número de accesos a disco necesarios para encontrar un registro.

Comportamiento Dinámico

La densidad de empaquetamiento no es constante; varía a medida que se insertan o eliminan registros del archivo.

Un archivo recién creado tendrá una DE muy baja. A medida que se incorporan nuevos registros, la DE aumenta progresivamente.

Cuando la DE alcanza un umbral elevado (por ejemplo, el 75% o más), la probabilidad de saturación aumenta considerablemente, lo que puede llevar a una necesidad de redispersar el archivo completo en un espacio de direcciones mayor. Este proceso de redispersión es muy costoso en términos de tiempo y recursos, ya que implica mover todos los registros y adaptar la función de hash.

Conexión con Hashing Estático y Dinámico

La DE es un concepto central en el hashing con espacio de direccionamiento estático, donde el espacio disponible para los registros se fija de antemano.

Las limitaciones asociadas con una DE elevada y el costo de la redistribución llevaron al desarrollo del hashing con espacio de direccionamiento dinámico (como el hash extensible), donde el espacio de almacenamiento puede crecer o disminuir según las necesidades, evitando así la necesidad de una DE predefinida y costosas redistribuciones completas. En estos métodos dinámicos, el concepto de DE ya no se aplica de la misma manera porque el espacio se ajusta automáticamente.

Problemas

Colisión

Situación en la que un registro es asignado, por función de dispersión, a una dirección que ya posee uno o más registros.

Desborde

Situación en la cual una clave carece de lugar en la dirección asignada por la función de dispersión.

Relación con Colisiones y Overflow (desborde)

La DE es crucial porque cuanto mayor sea la Densidad de Empaquetamiento, mayor será la posibilidad de que ocurran colisiones y, por ende, desbordes (overflow). Esto se debe a que hay menos espacio "vacío" para esparcir los registros.

Por el contrario, si la DE se mantiene baja, hay más espacio disponible, lo que disminuye la probabilidad de colisiones.

Estudio de la ocurrencia y probabilidad de Overflow

El estudio de la ocurrencia de desbordes (overflow) en dispersión (hashing) es crucial para evaluar la eficiencia de acceso a los registros y predecir cuándo no se puede garantizar un acceso directo y rápido.

Los puntos clave son:

- **Propósito:** Se analiza la probabilidad para entender la eficiencia del método de dispersión y cuándo un registro puede requerir más de un acceso a disco para ser localizado.
- **Herramienta:** Se utiliza la distribución de Poisson para modelar la probabilidad de que un nodo reciba i registros, dadas N direcciones de nodos y K registros a dispersar, con una capacidad C por nodo.
- **Factores Clave:** La probabilidad de overflow depende directamente de la Densidad de Empaquetamiento (DE) y de la capacidad (C) de cada nodo (cubeta).
 - Una mayor DE (más registros por espacio disponible) y una menor capacidad de nodo (C) incrementan significativamente la probabilidad de colisiones y desbordes.
 - Por el contrario, disminuir la DE (utilizando más espacio) y/o aumentar la capacidad de los nodos reduce la saturación, permitiendo que la mayoría de los registros se encuentren con un

solo acceso a disco. Por ejemplo, con una DE del 75%, si los nodos pueden almacenar 100 registros, la probabilidad de colisiones es inferior al 0.09%.

La siguiente tabla resume cómo la capacidad de los nodos y la Densidad de Empaquetamiento afectan la probabilidad de saturación:

DE	C=1	C=2	C=5	C=10	C=100
10%	4.8	0.6	0.0	0.0	0.0
50%	21.3	10.4	2.5	0.4	0.0
75%	29.6	18.7	8.6	4.0	0.0
100%	36.8	27.1	17.6	12.5	4.0

Se puede observar que, a medida que la capacidad de cada nodo aumenta, la probabilidad de saturación disminuye. Por ejemplo, con una DE del 75% y nodos que pueden almacenar hasta 100 registros, la probabilidad de colisiones es de menos de 0,09%.

Conclusión: Es un balance entre el uso eficiente del espacio en disco (DE alta) y la rapidez de acceso a los datos (minimizar overflows).

Técnicas de resoluciones

En el método de dispersión (hashing), un desborde u overflow ocurre cuando se intenta direccionar un registro a un nodo (o "cubeta") que ya no tiene capacidad para almacenarlo. Aunque la función de hash sea eficiente y la densidad de empaquetamiento (DE) sea relativamente baja, las colisiones (cuando dos o más claves compiten por la misma dirección física) pueden producir saturación. Por esta razón, es fundamental contar con métodos para reubicar los registros que no pueden ser almacenados en su dirección base original y asegurar que puedan ser encontrados posteriormente en su nueva ubicación.

Saturación progresiva

Este método consiste en almacenar el registro en la siguiente dirección disponible más próxima al nodo donde se produjo la saturación.

Funcionamiento

Si la dirección calculada por la función de hash está llena, se busca secuencialmente la siguiente posición libre en el archivo para guardar el registro. Por ejemplo, si el nodo 50 está lleno y se intenta insertar un registro allí, y el nodo 52 es el primero disponible después del 50, el registro se guarda en el nodo 52.

Búsqueda

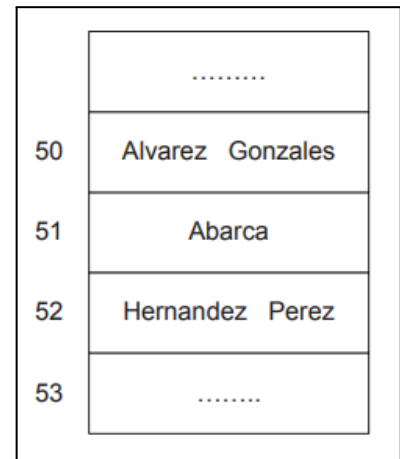
Para encontrar un registro, se calcula su dirección base con la función de hash. Si no se encuentra allí, la búsqueda continúa secuencialmente en los nodos subsiguientes hasta hallar el elemento o hasta encontrar un nodo que no esté completo (lo que indicaría que el elemento no está).

Desventajas

- Puede requerir revisar todas las direcciones disponibles en un caso extremo para localizar un registro, lo que hace la búsqueda potencialmente lenta.

- La eliminación de registros puede crear "huecos" que interrumpen la cadena de búsqueda. Si un registro que estaba en la dirección 51 es borrado, y la búsqueda de otro registro (Perez) empieza en 50 y luego va a 51, se detendría en 51 al encontrarlo vacío, sin llegar a Perez si este se reubicó más adelante. Para evitar esto, es necesario marcar los nodos que estuvieron saturados para que la búsqueda no se detenga prematuramente.

- Los registros en saturación tienden a agruparse en nodos cercanos, lo que puede provocar una saturación localizada excesiva y afectar el rendimiento de las búsquedas.



Saturación Progresiva Encadenada

Esta es una variante de la saturación progresiva. Un elemento que no cabe en su dirección base es ubicado en la siguiente dirección disponible, pero esta nueva dirección se enlaza con la dirección base inicial, creando una cadena de búsqueda.

Funcionamiento

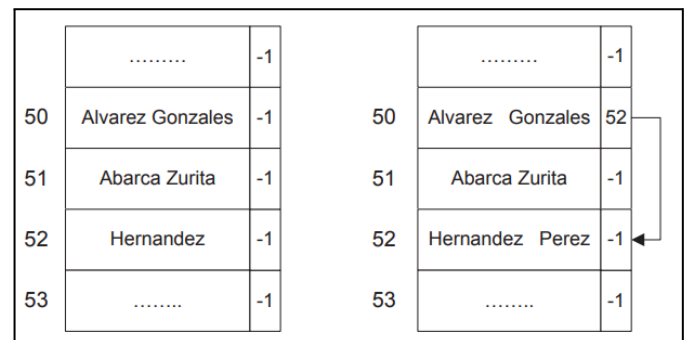
Cada nodo debe manejar información adicional: la dirección del siguiente nodo en la cadena de desborde. Si un registro como "Perez" causa un desborde en el nodo 50 y se guarda en el nodo 52, el nodo 50 ahora apuntará al nodo 52, formando una cadena.

Ventajas

Puede mejorar la performance de búsqueda respecto a la saturación progresiva simple al seguir una cadena directa en lugar de una búsqueda lineal. Por ejemplo, para encontrar "Perez", se podría pasar de tres accesos (con saturación progresiva) a solo dos (con encadenamiento).

Desventajas

Requiere que cada nodo manipule información extra (el puntero al siguiente nodo), aumentando el tamaño de la estructura.



Doble Dispersión

Este método utiliza dos funciones de hash para resolver colisiones y dispersar los registros en saturación a lo largo del archivo.

Funcionamiento

1. La primera función de hash (F1H) calcula la dirección base donde el registro debería ubicarse.
2. Si ocurre un overflow, se usa una segunda función de hash (F2H). Esta segunda función no devuelve una dirección, sino un desplazamiento.
3. Este desplazamiento se suma repetidamente a la dirección base obtenida por F1H hasta encontrar una dirección con espacio suficiente para el registro.

Ventajas

Tiende a esparcir los registros en saturación de manera más uniforme a lo largo del archivo de datos, evitando la concentración de desbordes en zonas contiguas.

Desventajas

Los registros en overflow tienden a ubicarse "lejos" de sus direcciones base, lo que puede aumentar el desplazamiento del cabezal lector/grabador del disco rígido (mayor latencia entre pistas) y, consecuentemente, el tiempo de respuesta.

Área de Desbordes por Separado

Para evitar que los registros en overflow ocupen direcciones que no les corresponden y afecten la performance del hashing, este método utiliza un área de desbordes separada y dedicada.

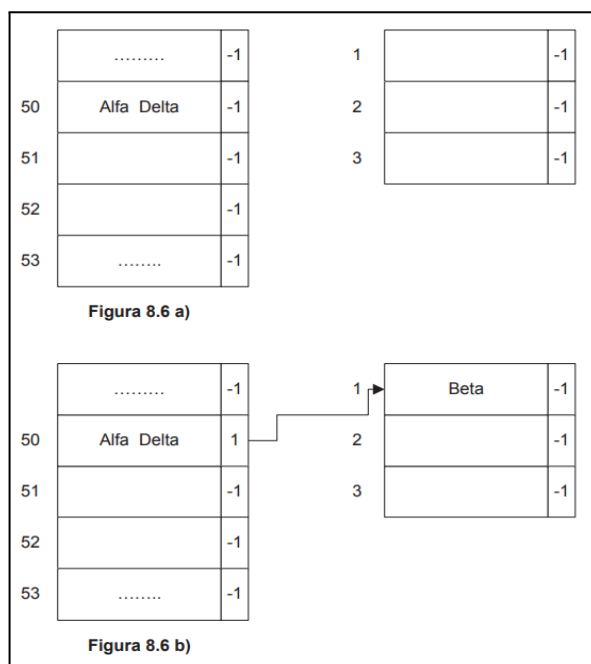
Funcionamiento

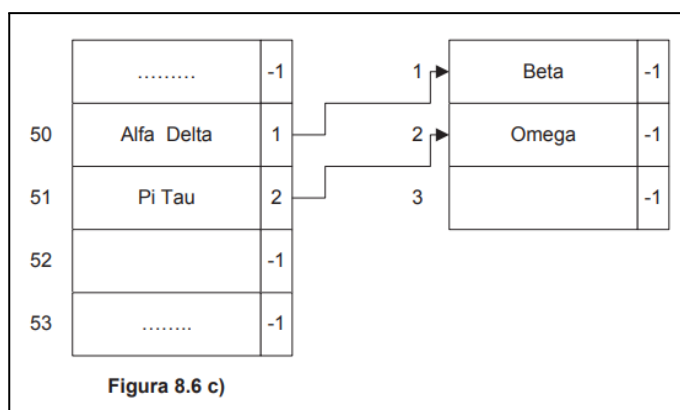
- Se distinguen dos tipos de nodos: aquellos direccionables por la función de hash (área principal) y aquellos de reserva (área de desbordes) que solo se usan en caso de saturación y no son directamente accesibles por la función de hash.

- Cuando un registro causa un desborde en el área principal, se reubica en la primera dirección disponible dentro de esta área de desbordes separada.

- La dirección base original en el área principal se encadena con la dirección del registro reubicado en el área de desbordes, estableciendo un vínculo directo. Si la cubeta de desbordes también se llena, el último nodo de la cadena se enlaza con un nuevo nodo en el área de desbordes.

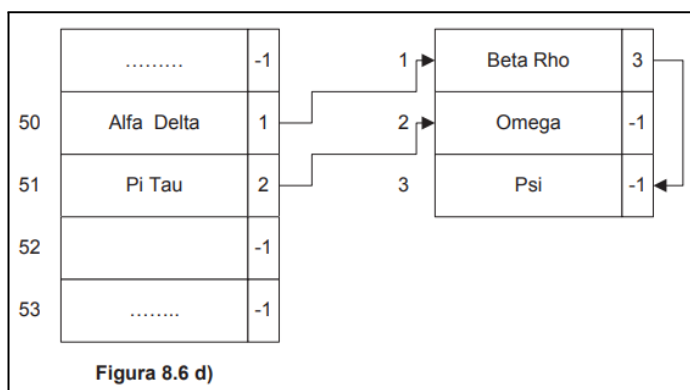
La **Figura 8.6 a)** presenta un ejemplo del problema. Las llaves Alfa y Delta direccionan la cubeta 50. Suponiendo que esta tiene capacidad para dos registros, se completa. Al dispersar la clave Beta, la dirección obtenida a partir





de la función de hash es nuevamente la dirección 50. Se produce saturación, y el registro es reubicado en la primera dirección disponible dentro del área de desbordes separada [**Figura 8.6 b)**]. La dirección base original se encadena con la dirección de reserva. Luego, como se muestra en la **Figura 8.6 c)**, se dispersan las llaves Pi y Tau, con dirección base 51; si llega Omega al mismo nodo, se produce overflow y se utiliza otra dirección del área de desbordes, la 2

en este caso. Por último, la **Figura 8.6 d)** muestra lo que ocurriría si llegaran dos claves más que obtuvieran la dirección 50. En este caso, Rho y Psi; la primera de ellas se ubica en la dirección 1 de desbordes; la segunda no cabe: por lo tanto, se redirecciona un nuevo nodo, el tercero del área de desbordes, el cual se enlaza con el primero.



Ventajas

Los registros en overflow no interfieren con la búsqueda de otros registros en el área principal, lo que puede mejorar el tratamiento de inserciones y eliminaciones. Permite que la mayoría de los registros residan en sus ubicaciones calculadas, manteniendo un acceso rápido.

Desventajas

Puede empeorar el Tiempo de Acceso Promedio (TAP) debido a que los registros de desborde se almacenan en un área diferente, que podría requerir un acceso adicional, y la búsqueda implicaría seguir punteros a esa área.

Hash asistido por tabla

El método de hash asistido por tabla es una variante del hash con espacio de direccionamiento estático que asegura el acceso a un registro de datos en una sola operación de disco. A diferencia de otras técnicas de hash que no requieren almacenamiento adicional, esta necesita una estructura adicional: una tabla que se administra en memoria principal.

Este método utiliza tres funciones de hash:

- F1H: Retorna la dirección física del nodo donde el registro debería residir.
- F2H: Retorna un desplazamiento (similar al método de doble dispersión).
- F3H: Retorna una secuencia de bits que no pueden ser todos unos.

Proceso de Inserción

Comienza con la F1H. Si el nodo resultante tiene espacio, el registro se almacena allí. Si se produce una saturación (overflow), el procedimiento es más complejo:

1. Se obtiene el valor de F3H para todos los registros en el nodo saturado.
2. Se determina la clave que genera el valor F3H más alto.
3. El nodo original se reescribe con todos los registros excepto el que tiene el F3H mayor.
4. En la tabla en memoria, para la posición F1H original, se guarda el valor F3H de la clave que fue "quitada" del nodo.
5. Para la clave seleccionada en el paso 2, se obtiene su F2H y se suma a su F1H para obtener una nueva dirección.
6. Se intenta insertar la clave en esta nueva dirección. Si hay saturación nuevamente, el proceso se repite desde el paso 1.

Proceso de Recuperación (Búsqueda)

1. Se calcula F1H y F3H para la clave buscada.
2. Se compara el valor F3H de la clave con el valor almacenado en la entrada de la tabla indicada por F1H.
 - Si el F3H de la clave es menor que el valor en la tabla, se busca directamente en la dirección del nodo indicada por F1H y el proceso termina.
 - Si el F3H es mayor o igual, se calcula un desplazamiento usando F2H y se suma a F1H para obtener una nueva dirección. El proceso se repite desde el paso 3 con la nueva posición. Este método asegura encontrar la clave buscada en un solo acceso a disco, ya que las operaciones adicionales (cálculo de F2H y F3H, y consulta de la tabla) se realizan en memoria principal, lo cual tiene un costo bajo comparado con el acceso a disco.

Proceso de Eliminación (Baja)

El proceso de eliminación con hash asistido por tabla es relativamente simple:

1. Se localiza el registro utilizando el proceso de búsqueda descrito.
2. Si se encuentra la llave, el nodo se reescribe sin el elemento a borrar. Es importante considerar un caso especial: si se borra una clave de un nodo que antes estaba completo y ya había provocado un overflow, la entrada en la tabla en memoria para ese nodo contendrá el valor F3H de la clave que fue reubicada debido a la saturación. Si se inserta un nuevo elemento en ese nodo, su F3H debe ser menor que el valor en la tabla para esa dirección, para asegurar que la búsqueda futura de registros reubicados no se vea impedida.

Hash con Espacio de Direccionamiento Dinámico

El hash con espacio de direccionamiento estático, donde la cantidad de direcciones disponibles se fija de antemano, presenta problemas cuando el archivo crece significativamente:

- Alcanzar una Densidad de Empaquetamiento (DE) alta (por ejemplo, 75% o más) aumenta considerablemente la probabilidad de saturación (overflow).
- Cuando el archivo se "llena", es necesario aumentar la cantidad de direcciones disponibles y, por ende, modificar la función de hash, lo que obliga a redispersar (reubicar) todos los registros ya almacenados. Este proceso es muy costoso en tiempo y no permite el acceso al archivo durante su ejecución.

Para evitar la costosa redistribución completa, surge la necesidad de administrar el espacio de direccionamiento de manera dinámica. En este enfoque, la cantidad de nodos disponibles crece o disminuye en función de las necesidades del archivo en cada momento, sin estar predefinida. Varias alternativas de implementación existen para este tipo de hash, incluyendo hash virtual, hash dinámico y hash extensible.

Hash Extensible

El método de hash extensible es una implementación del hash con espacio de direccionamiento dinámico. Su principio fundamental es empezar con un único nodo de almacenamiento y aumentar progresivamente la cantidad de direcciones disponibles a medida que los nodos se van llenando.

Características clave

- No utiliza el concepto de Densidad de Empaquetamiento (DE) porque el espacio en disco se ajusta dinámicamente.
- La función de hash no devuelve una dirección física fija, sino una secuencia de bits (por ejemplo, 32 bits), y la cantidad de bits utilizados en un momento dado determina el número máximo de direcciones a las que el método puede acceder.
- Requiere una estructura auxiliar: una tabla en memoria principal. A diferencia del hash asistido por tabla, esta tabla contiene directamente la dirección física de cada nodo. La secuencia de bits de la función de hash se usa para obtener la dirección física del nodo desde esta tabla.

Proceso de Inserción

1. Estado inicial: El archivo comienza con un solo nodo en disco y una tabla en memoria con una única entrada, asociada a un valor de 0 bits, indicando que no se necesita ningún bit de la secuencia de hash para direccionar.
2. Inserción sin overflow: Las claves se insertan en el nodo disponible si hay capacidad.
3. Overflow y duplicación de tabla: Si una inserción causa overflow en un nodo:
 - El valor asociado al nodo saturado se incrementa en uno, y se genera un nuevo nodo con el mismo valor.
 - Si el valor del nodo saturado es ahora mayor que el valor asociado a la tabla (indicando que la tabla actual no tiene suficientes entradas para direccionar todos los nuevos nodos), la cantidad de celdas de la tabla se **duplica**, y el valor asociado a la tabla se incrementa en uno (lo que implica tomar un bit adicional de la función de hash).
 - Los elementos del nodo saturado (los antiguos y el que causó el overflow) son redistribuidos entre el nodo viejo y el nuevo nodo según el valor del bit menos significativo (el bit adicional) de su función de hash.
 - Overflow **sin duplicación** de tabla: Si al producirse un overflow, el valor asociado al nodo saturado ya coincide con el valor asociado a la tabla, significa que la tabla ya tiene suficientes entradas para direccionar el nuevo nodo. En este caso, no se duplica la tabla; simplemente se realiza la dispersión de los elementos entre el nodo viejo y el nuevo nodo utilizando un bit adicional de la función de hash.

Proceso de Búsqueda

- Se calcula la secuencia de bits para la llave buscada.
- Se toman tantos bits de esa secuencia como indique el valor asociado a la tabla en memoria.
- La dirección del nodo contenida en la celda respectiva de la tabla debería contener el registro buscado.
- Si el registro no se encuentra en ese nodo, significa que el elemento no forma parte del archivo de datos. El hash extensible asegura encontrar cada registro en un solo acceso a disco. El costo asociado es un mayor procesamiento cuando una inserción genera un overflow y requiere la duplicación de la tabla y la redistribución de elementos.

Conclusión general

Organización	Acceso a un registro por clave primaria	Acceso a todos los registros por clave primaria
Ninguna	Muy ineficiente	Muy ineficiente
Secuencial	Poco eficiente	Eficiente
Secuencial indizada	Muy eficiente	El más eficiente
Hash	El más eficiente	Muy ineficiente

Archivos (Organización Básica: Ninguna o Secuencial)

Los archivos son la capa fundamental donde la información se almacena de forma persistente en dispositivos de almacenamiento secundario, como los discos rígidos.

• **Organización "Ninguna" (Secuencial Física):** En este modo, los registros se guardan en el orden en que se ingresan, sin ningún criterio lógico preestablecido más que su ubicación física.

◦ Acceso a un registro por clave primaria: Es muy ineficiente, ya que para encontrar un dato específico, se deben recorrer todos los registros previos en el orden en que fueron almacenados. En el peor de los casos, esto implica N accesos (lecturas), siendo N la cantidad de registros, resultando en una performance de orden $O(N)$.

◦ Acceso a todos los registros por clave primaria: También es muy ineficiente si se necesita un orden lógico que no corresponde con el orden físico.

◦ Buffering: Las operaciones de lectura y escritura se realizan sobre buffers (memoria intermedia en RAM), lo que optimiza la transferencia de datos minimizando el número de accesos al disco.

Archivos Secuenciales Indizados (Árboles como Índices)

Este método utiliza una estructura de datos auxiliar, llamada índice, para organizar y acelerar el acceso a la información en un archivo, sin necesidad de reordenar físicamente el archivo de datos.

- **Propósito:** Un índice es una tabla que contiene claves (llaves de búsqueda) y referencias (direcciones) para encontrar el registro correspondiente dentro del archivo de datos, permitiendo imponer un orden lógico en el archivo. Esto proporciona múltiples caminos de acceso a un archivo.
- **Acceso a un registro por clave primaria:** Es muy eficiente. La búsqueda se realiza primero en el índice (que es más pequeño y, a menudo, cabe en memoria principal o puede manejarse con pocos accesos a disco), y luego se accede directamente al registro de datos. La performance de búsqueda puede ser de orden logarítmico ($\log_2(N)$).
- **Acceso a todos los registros por clave primaria:** Es el más eficiente, especialmente con estructuras como los árboles B+ que permiten un recorrido secuencial de todos los nodos terminales.

Estructuras de árboles balanceados

- **Árboles B:** Son árboles multicamino que garantizan que todas las hojas (nodos terminales) estén al mismo nivel, manteniendo el árbol balanceado a bajo costo. Cada nodo puede tener múltiples elementos y descendientes. La eficiencia de búsqueda en un árbol B se mide por la altura del árbol (H lecturas), siendo H un valor pequeño incluso para archivos muy grandes (ej. 4 accesos para 1 millón de registros). Las operaciones de inserción y eliminación también son eficientes.
- **Árboles B*:** Una variante de los árboles B que busca maximizar la ocupación de los nodos (al menos $2/3$ llenos) mediante redistribuciones entre hermanos antes de dividirse, lo que desacelera el crecimiento de la altura del árbol. La búsqueda es similar a los árboles B.
- **Árboles B+:** Diferencian los elementos que constituyen datos de aquellos que son separadores. Toda la información de los datos se encuentra únicamente en los nodos terminales, los cuales están enlazados secuencialmente. Los nodos internos contienen solo claves que actúan como "separadores" o puntos de referencia. Esto permite búsquedas aleatorias rápidas y un acceso secuencial muy eficiente al recorrer los nodos terminales enlazados. Los árboles B+ de prefijos simples optimizan el uso del espacio al usar la mínima expresión posible de la clave como separador.

Dispersión (Hashing)

La dispersión (hashing) es una técnica que busca el acceso directo a un registro en un único acceso a disco en promedio. A diferencia de los índices basados en árboles, el hashing organiza directamente el archivo de datos y no requiere una estructura auxiliar para el acceso rápido.

- **Funcionamiento:** Una función de hash transforma la clave primaria de un registro en una dirección física donde almacenar dicho registro.
- **Ventajas:** Permite inserción y eliminación muy rápidas. Localiza registros con un solo acceso a disco en promedio.

Limitaciones

- No es posible usarlo con registros de longitud variable.
- No mantiene un orden lógico de los datos; los registros se "esparcen" aleatoriamente.
- No puede trabajar con claves duplicadas (claves secundarias).

• **Colisiones y Desbordes (Overflow):** El problema principal son las colisiones (varias claves mapean a la misma dirección) y el desborde (un nodo se llena). La Densidad de Empaquetamiento (DE) (registros / capacidad total) influye en la probabilidad de overflow: a menor DE, menor overflow pero mayor desperdicio de espacio.

• **Métodos de Tratamiento de Overflow (Hash Estático):** Incluyen saturación progresiva, saturación progresiva encadenada, doble dispersión y área de desbordes por separado. Estos métodos buscan reubicar registros en caso de colisión.

• **Hash Asistido por Tabla:** Una variante de hash estático que, al usar una tabla adicional en memoria principal y múltiples funciones de hash, asegura encontrar el registro en un solo acceso a disco. Sin embargo, implica complejidad adicional en inserciones con saturación.

• **Hash con Espacio de Direccionamiento Dinámico (Hash Extensible):** Permite que el espacio de direcciones disponible para los registros crezca o disminuya dinámicamente según las necesidades, evitando la costosa redistribución completa de todo el archivo que ocurre en el hash estático. Este método utiliza una tabla en memoria principal que mapea una secuencia de bits (resultado de la función de hash) a las direcciones físicas de los nodos, garantizando un único acceso a disco para la búsqueda.

La elección del método de organización de archivos dependerá de los requisitos específicos del sistema, como la frecuencia de las operaciones de búsqueda aleatoria vs. secuencial, la volatilidad de los datos y las limitaciones de espacio.